

Road Trip Cost Prediction

Predicting the cost of a road trip between any two Indian cities — and being honest about what the model actually knows.

Author	Param Kotadiya
Subject	Machine Learning (Semester 5)
Artifact	A deployed web application, and the scripts that fit every model inside it
Problem type	Regression (primary) + 1 classification task
Target	<code>total_trip_cost</code> (continuous, ₹)
Datasets	1,140 original trips · 25,000 generated trips · 840 real road distances
Models trained	8 models across 6 algorithm families, scored on one split
Tests	70 automated tests, all passing

The one thing to take from this report

It reports **two** accuracy figures, and knowing why is the whole point of it.

$R^2 = 0.999778$ is real, but it was measured with the true `toll_cost` and `fuel_consumption_litres` handed to the model as input features. Those are the two terms that carry the uncertainty in the total, and a traveller knows neither before setting off. It answers “*given the litres burnt and the toll paid, can you total the bill?*”

$R^2 = 0.9816$, MAE ₹292 is what the model achieves predicting from what a user actually supplies: a distance, a fuel price and a parking figure. That is the number the deployed application reports, and the honest one to quote.

CONTENTS

What is in this report

This document is organised **by topic**, not by the order the work happened in. Each part covers one subject — the data, the preprocessing, the regression, the feature engineering, the models, the evaluation — and can be read on its own. Part 10 is the centre: every model in the project, what it takes in, what it returns, and what it scores.

Part 1	The project at a glance — problem, stack, and the model catalogue	overview
Part 2	The dataset — every column, every statistic	data
Part 3	Exploring the data	data
Part 4	Data preprocessing — all eight steps	data
Part 5	Recovering the generating formula	analysis
Part 6	Building the training data and the distance table	data
Part 7	Linear regression: <code>fit()</code> and gradient descent from scratch	modelling
Part 8	Feature engineering — the interaction terms	modelling
Part 9	Regularisation and ill-conditioning	modelling
Part 10	The model pipeline — every model and what it does	models
Part 11	Classification — the cost band	models
Part 12	Evaluation, validation and tuning	evaluation
Part 13	The application — API, frontend, deployment, tests	engineering
Part 14	Bugs found in this project's own code, and limitations	honesty
Part 15	Glossary of every technical term used	reference
Part 16	How to reproduce every number in this report	reference

A note on how to read the tables

Wherever a model is scored, the report gives **train** and **test** figures side by side, plus a **cross-validation mean and standard deviation**. A single test score can be luck; the CV standard deviation is what says whether a model is stable. Every model is also shown against a **baseline row** — the simplest thing that could stand in its place. A model that cannot beat that has not earned its slot, and one of them does not.

The project at a glance

1.1 The problem

Road trips involve several variable costs — fuel, tolls, parking, wear on the vehicle — which depend on the route, the vehicle, traffic conditions and how far you are driving. Estimating that total in advance is useful for planning and budgeting.

Item	Value
Problem statement	Given the details of a road trip, predict the total trip cost using machine learning.
Target variable	<code>total_trip_cost</code> — continuous, in rupees
Problem type	Regression (a continuous numeric value). Two classification tasks were added in Part 11 where they add value.
Original dataset	<code>road_trip_data.csv</code> — 1,140 rows, 16 columns, 9 fixed city pairs
Final serving domain	Any two of 537 Indian cities (population $\geq$ 100,000), 50–1,500 km

1.2 What the finished thing does

The user types **how far they are driving**, what fuel costs where they are, and what they expect to pay for parking. Back comes a price for a **hatchback, a sedan and an SUV at once**, on all three fuels, each with a full cost breakdown, an efficiency band, an honest error bar and the caveats that apply to that particular estimate.

They are asked only for things they could actually know. Everything else — the litres they will burn, the tolls they will pay, and the km/l each vehicle manages — is **predicted by its own fitted sub-model**, and a chained cost regressor totals them. Distance is the one lookup, and the site ships a table of measured road distances between 139 cities so it is not a lookup the user has to do elsewhere.

1.3 Every model in this project, and what it does

Eight fitted things carry the product. Five are sub-models that supply a quantity the user cannot, one is the regressor that totals them, one is the label on each result, and one is a method demonstration rather than a served model. **Part 10 scores all of them**; this table is the map.

Model	In → out	Family	What it is for
Distance	four coordinates → road km	RandomForest regressor	Answers the distance lookup for any city pair the shipped table does not carry, and whenever live routing is unreachable. The only model here trained on observed rather than generated data .
Mileage	vehicle + fuel → km/l	Linear regression	Gives each of the three vehicles its own fuel efficiency. One typed figure cannot be honest for a hatchback and an SUV at once, and the gap between the three is most of the gap between their costs.
Litres	distance + mileage → litres	Linear regression	How much fuel the trip burns. Its fitted slope sits above the textbook $\text{distance} \div \text{mileage}$, because real driving costs more than the rated figure.
Toll	distance → ₹	Linear regression	The toll bill. Close to a per-km rate, but fitted and carrying its own error bar rather than asserted as a constant.
Parking	—	none — it failed	Six model families were fitted and every one scored worse than predicting the mean. That negative result is why parking is the one cost the form asks for outright.
Cost (chained)	the four above + what you typed → ₹	Linear regression	The headline. Trained on its sub-models' <i>predictions</i> rather than on ground truth, so training and serving see the same kind of input. Called three times per request, once per vehicle.
Cost band	the cost terms → Excellent / Good / Average / Poor	Logistic regression	Is this trip cheap or expensive <i>for its length</i> ? A quartile split of ₹/km, which is the one judgement the rupee figure cannot make on its own.
Gradient descent	same fit, solved iteratively	from scratch, NumPy only	Not served. It solves the same regression as the closed form and is kept to show the two agree on predictions while disagreeing on weights — which is what an ill-conditioned problem looks like.

The shape of the thing: what is asked for, and what is derived

The user supplies four numbers — **distance, fuel price, parking, passengers** — and gets back a cost for a hatchback, a sedan and an SUV, on all three fuels, with a breakdown for each.

Everything else is predicted, and the split is deliberate. **Distance** is asked for because it is the one input a traveller can look up exactly; the site ships a table of measured road distances so they do not have to go elsewhere. **Parking** is asked for because nothing predicts it. **Litres and tolls are never asked for**, because nobody knows them before setting off — and litres is the cost model's single strongest feature, so requesting it would be handing the model its own answer.

That last point is the whole design. An input is only worth collecting if the user can actually answer it.

1.4 Technology used

Layer	Choice	Why
Language	Python 3.13	Standard for ML coursework
Data	pandas 3.0.3, NumPy	Loading, cleaning, tabular manipulation
Models	scikit-learn 1.9.0	LinearRegression, Ridge, DecisionTree, RandomForest, AdaBoost, GradientBoosting, LogisticRegression
From scratch	NumPy only	Gradient descent, so Part 7 shows what <code>fit()</code> does underneath
Charts	Recharts	The loss curve and the sub-model story, drawn in the browser from <code>/api/loss-curve</code>
Backend	FastAPI + uvicorn	Serves the models over HTTP
Frontend	React 19, Vite 7, Tailwind v4, GSAP	Single-page editorial site
Routing data	OSRM public API	Real road distances
Geography	GeoNames <code>cities15000</code> (CC BY 4.0)	3,739 Indian cities with real latitude/longitude
Testing	pytest	70 API and model-behaviour tests
Deployment	Render (API) + Vercel (UI)	A Python process and a static build, deployed separately — Part 13.6

1.5 File map

README.md	Front door: what it does, how to run it, how to deploy it
render.yaml	Render blueprint - names backend/ as the service root
backend/	DEPLOYED to Render. Holds only what answers a request.
app.py	FastAPI - the chained prediction pipeline
roadtrip_features.py	The feature contract, imported by the API AND by training
requirements.txt	Exact versions: the model files are pickles (Part 13.6)
Dockerfile	For any container host; Render uses the blueprint instead
data/	
india_cities.csv	3,739 cities, real coordinates (GeoNames)
fuel_prices.csv	35 states; a `source` column marks verified vs estimated
models/	
model.joblib	Cost-band classifier + gradient-descent history (10 KB)
pipeline.joblib	Sub-models + chained cost regressor (5.1 MB, served)
tests/test_api.py	70 tests
frontend/	DEPLOYED to Vercel. React 19, Vite 7, Tailwind v4, GSAP.
src/api.js	Every URL in the project, in one file
public/city_distances.json	5,538 measured routes - shipped WITH the UI, so the lookup survives the API being asleep
vercel.json	Build settings
ml/	DEPLOYED NOWHERE. Everything that produced the models.
road_trip_data.csv	The original 1,140 trips. Never modified.
Project_Report.pdf	This document
report/	Its HTML sources; two chapters are generated, not typed
data/	
road_trip_wide.csv	25,000 generated trips over 534 cities
real_distances.csv	840 REAL OSRM road distances - the only observed data
pipeline_report.json	Saved metrics for every sub-model
regression_report.json	Saved metrics for Parts 7-9
task5_evaluation.json	Every Task 5 number
scripts/	
evaluation.py	The Task 5 checklist, implemented once
build_wide_dataset.py	Regenerates the training data from the recovered formula
fetch_real_distances.py	Collects the 840 real road distances from OSRM
train_pipeline.py	Fits and scores the sub-models -> backend/models/
train_models.py	Fits the band classifier and the curve -> backend/models/
evaluate_models.py	Runs the whole Task 5 sweep
build_report_pdf.py	Assembles report/ into the PDF

Why there is a shared `roadtrip_features.py` at the project root

Every model is trained in `scripts/` and served from `app.py`. If those two built their feature vectors independently they would drift — a column reordered in one place and not the other gives predictions that are wrong but not *obviously* wrong. So each model has exactly one function that returns its features in a fixed order, and both sides call it. It sits at the project root rather than in `scripts/` because the Docker image excludes `scripts/`, so the API could not import from there.

PART 2

The dataset

2.1 Every column of the original dataset

road_trip_data.csv — 1,140 rows × 16 columns, 181 KB in memory. Eight float64, two int64, six text columns.

Column	Type	Meaning	Corr. with target
start_city	str	Origin city. Only 9 fixed pairs exist in this file.	–
destination_city	str	Destination city.	–
distance_km	float	Road distance of the trip.	+0.8315
vehicle_type	str	Hatchback, Sedan or SUV.	–
fuel_type	str	Petrol, Diesel or CNG.	–
mileage	float	Vehicle fuel efficiency, km per litre.	–0.3802
fuel_price	float	Price per litre on the day of travel.	+0.1450
passengers	int	Number of people on the trip, 1–5.	–0.0468
departure_hour	int	Hour of departure, 0–23.	–0.0261
traffic_level	str	Low, Medium or High.	–
weather_condition	str	Clear, Cloudy, Fog or Rain.	–
toll_cost	float	Tolls paid. A direct additive component of the bill.	+0.6701
parking_cost	float	Parking paid. Another direct additive component.	+0.0970
estimated_time_minutes	float	Journey duration.	+0.7282
fuel_consumption_litres	float	Litres actually burnt. The single strongest driver.	+0.9606
total_trip_cost	float	TARGET — the total bill in rupees.	1.0000

2.2 Descriptive statistics

Column	mean	std	min	25%	50%	75%	max
distance_km	239.77	85.06	110.13	158.46	263.48	286.07	459.64
mileage	15.45	3.27	8.18	12.98	15.47	17.91	23.12
fuel_price	93.04	8.96	72.13	88.09	94.79	98.92	107.99
passengers	3.07	1.41	1	2	3	4	5
departure_hour	11.58	6.87	0	6	12	18	23
toll_cost	303.64	116.54	99.47	215.75	289.57	381.89	665.22
parking_cost	68.86	45.32	0.00	40.00	60.00	120.00	150.00
estimated_time_minutes	246.83	67.95	95.63	204.00	254.25	291.66	—

Target range across the cleaned 1,129 rows: ₹688.79 to ₹4,271.82.

2.3 Categorical values, and the 9 fixed routes

Column	Distinct values
vehicle_type	Sedan, Hatchback, SUV — 3 values
fuel_type	Petrol, Diesel, CNG — 3 values
traffic_level	Low, Medium, High — 3 values
weather_condition	Clear, Cloudy, Fog, Rain — 4 values

The nine routes, with their straight-line distance and the actual road distance — this table becomes important in Parts 6 and 10:

Route	Straight line (km)	Road (km)	Winding factor
Ahmedabad → Surat	205.0	274.6	1.340
Ahmedabad → Udaipur	207.9	271.3	1.305
Ahmedabad → Vadodara	102.9	116.9	1.136
Bengaluru → Mysuru	127.8	150.9	1.180
Delhi → Jaipur	239.3	288.9	1.207
Pune → Mumbai	119.5	161.2	1.349
Rajkot → Ahmedabad	201.4	223.7	1.111
Surat → Mumbai	236.1	302.4	1.280
Vadodara → Mumbai	360.4	436.1	1.210
Winding factor across these 9 routes			mean 1.2355, sd 0.0820

2.4 The three datasets, and which one is real

Dataset	Rows	What it is	Real or generated?
road_trip_data.csv	1,140	The dataset handed out with the assignment. 9 city pairs, 110–460 km.	Synthetic (proved in Part 5)
road_trip_wide.csv	25,000	The same cost physics regenerated over real Indian geography, 534 cities, 50–1,500 km.	Generated (Part 6), on real coordinates
real_distances.csv	840	Actual driving distances from the OSRM routing engine for a stratified sample of city pairs.	Observed — the only real data

Be ready for this question

"If your data is synthetic, what have you actually learned?"

Three answers, all defensible. First, the synthetic nature was **discovered, not assumed** — Part 5 recovered the generating formula from the data with R^2 0.999162, which is itself a result. Second, the geography is **real**: cities, coordinates and 840 road distances come from GeoNames and OSRM, so the distance sub-model is trained on observed data. Third, the project states explicitly which numbers are measurements of real skill and which are measurements of formula recovery — that distinction is Part 10 of this report and the reason two accuracy figures are quoted rather than one.

Exploring the data

3.1 What Part 3 did

1. Loaded `road_trip_data.csv` and inspected the first rows.
2. Stated the problem and identified the target as continuous → a regression problem.
3. Produced `df.shape`, `df.info()` and `df.describe()`.
4. Checked for missing values, duplicates, and listed the distinct value of every categorical column.

3.2 Findings

Check	Result
Shape	1,140 rows × 16 columns
Missing values	Zero in every one of the 16 columns (0.0%)
Duplicate rows	Zero
Data types	8 float64, 2 int64, 6 text — all already correct
Target	<code>total_trip_cost</code> , continuous → regression

Why “no missing values” was itself a clue

A real-world dataset of 1,140 road trips with *zero* missing values, zero duplicates and perfectly consistent dtypes is unusual. At the time this was recorded as a convenience. In Part 5 it became evidence: datasets this clean are usually generated. Noticing it early and testing it later is the honest version of the story.

3.3 The correlation analysis that drove feature selection

Every numeric column was correlated against the target. This single table decided which features Part 7 would use.

Feature	Pearson r	Interpretation
<code>fuel_consumption_litres</code>	+0.9606	Dominant. Burning fuel is the main cost.
<code>distance_km</code>	+0.8315	Strong. Longer trip → more fuel.
<code>estimated_time_minutes</code>	+0.7282	Strong, but nearly a copy of distance.
<code>toll_cost</code>	+0.6701	A direct additive component of the bill.
<code>fuel_price</code>	+0.1450	Weak alone — it only matters <i>multiplied</i> by litres. Part 8 exploits this.
<code>parking_cost</code>	+0.0970	Additive but small and random.
<code>departure_hour</code>	-0.0261	Effectively zero.
<code>passengers</code>	-0.0468	Effectively zero.
<code>mileage</code>	-0.3802	The only meaningful negative — an efficient car costs less to run.

The insight hiding in this table

`fuel_price` correlates only **+0.145** with cost, which looks unimportant. But fuel is the largest single cost, and the real fuel bill is `litres × price`. Price appears weak on its own because it only matters *as a product* with litres — and a linear model can only add its features, never multiply them. Part 8 turns this observation into the project's biggest single accuracy gain.

PART 4

Data preprocessing — all eight steps

Part 4 followed the standard 8-step preprocessing checklist. Each step is given below with what was done, why, and what changed.

Step 1 — Understand the data

Re-ran `shape`, `head()` and `info()` to confirm the starting point: 1,140×16, no nulls.

Step 2 — Handle missing values

Counted nulls per column both as a count and as a percentage. **Result: zero missing in all 16 columns.** No imputation was needed, so none was invented.

What would have been done otherwise: numeric columns → median (robust to outliers, unlike the mean); categorical columns → mode; a column missing more than about half its values → drop it rather than fabricate it. Being able to state the policy you *would* have used matters more in a viva than the fact that this dataset needed none.

Step 3 — Remove duplicate rows

`df.duplicated().sum()` = 0 before and after `drop_duplicates()`. Shape unchanged at 1,140×16.

Step 4 — Fix data types

Printed `df.dtypes` and explicitly searched for columns stored as text that ought to be numeric or dates. **Result: none.** The six text columns are genuinely categorical.

Step 5 — Handle outliers

Boxplots were drawn for all ten numeric columns, then the **IQR rule** was applied to each in turn:

$$IQR = Q3 - Q1 \cdot \text{keep rows where } Q1 - 1.5 \times IQR \leq x \leq Q3 + 1.5 \times IQR$$

Measure	Value
Rows before outlier removal	1,140
Rows after outlier removal	1,129
Rows dropped	11 (0.96%)

Boxplots were redrawn afterwards to confirm the extreme values were gone. Losing 11 rows out of 1,140 is a cheap price for removing points that would otherwise drag a least-squares fit.

An honest caveat on this step

The filter is applied column by column in a loop, so a row is dropped if it is extreme on *any* single column. That is stricter than a multivariate outlier test and it is applied **before** the train/test split, which technically lets information about the test rows influence which rows exist. With 11 rows out of 1,140 the effect is negligible, but the correct order in a strict pipeline is split first, then fit any data-dependent transformation on the training half only. Parts 10 and 12 do exactly that for scaling — see Part 14.

Step 6 — Encode categorical columns

A model needs numbers. The six text columns were **one-hot encoded** with `pd.get_dummies(..., drop_first=True)`.

Item	Value
Columns encoded	<code>start_city</code> , <code>destination_city</code> , <code>vehicle_type</code> , <code>fuel_type</code> , <code>traffic_level</code> , <code>weather_condition</code>
Shape after encoding	1,129 rows × 31 columns (from 16)
Method	One-hot, because none of these categories is <i>ordinal</i>

Why one-hot and not label encoding. Label encoding would map Hatchback→0, Sedan→1, SUV→2, which tells the model that an SUV is “twice” a Sedan and that Sedan sits exactly between the other two. None of that is true. One-hot gives each category its own independent column and asserts no ordering. `drop_first=True` removes one column per group to avoid the *dummy variable trap* — if you keep all three vehicle columns they sum to 1 in every row, which is perfectly collinear with the intercept.

Step 7 — Scale numeric columns

`distance_km` runs into the hundreds while `passengers` is 1–5. Features on wildly different scales distort distance-based and gradient-based methods, so **MinMaxScaler** mapped every numeric feature into 0–1:

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Nine columns were scaled: `distance_km`, `mileage`, `fuel_price`, `passengers`, `departure_hour`, `toll_cost`, `parking_cost`, `estimated_time_minutes`, `fuel_consumption_litres`.

One deliberate decision worth quoting

The **target was left unscaled**. That means every learned coefficient reads directly as “rupees of cost per unit of this feature”, and because the features are all on 0–1, a weight is literally “how many rupees the cost changes as this feature moves from its minimum to its maximum”. Scaling the target would have made the model equally accurate and completely unreadable.

Why MinMax and not StandardScaler? Standardisation (subtract mean, divide by standard deviation) assumes a roughly Gaussian spread and produces unbounded values. MinMax gives a bounded 0–1 range, which suits gradient descent — all partial derivatives end up on a comparable scale, so a single learning rate works for every weight.

Step 8 — Visualise

Chart	Column	What it showed
Histogram	<code>total_trip_cost</code>	Distribution of the target — roughly unimodal, right-leaning
Box plot	<code>distance_km</code>	Median, quartiles and remaining spread after cleaning
Scatter plot	distance vs cost	The clear positive relationship the model exploits
Line plot	cost over row order	A proxy for trend over time; no drift visible
Bar chart	<code>vehicle_type</code>	Trip counts per vehicle
Pie chart	<code>fuel_type</code>	Share of each fuel
Count plot	<code>traffic_level</code>	Class balance for the Part 11 classifier
Boxplot grid ×2	all 10 numeric	Before and after outlier removal

Part 4 output

The chain **cleaned** → **de-duplicated** → **outlier-trimmed** → **one-hot encoded** → **MinMax-scaled** produced `df_scaled`: 1,129 rows × 31 columns, which is where Part 7 begins.

Recovering the generating formula

5.1 The question that started it

Part 7 ended at $R^2 = 0.9928$. That is a suspiciously good score for a dataset with weather, traffic and human behaviour in it. So Part 5 asked a different question — **not “how well can we predict the target” but “what actually generated it?”**

The method is deliberately simple: **subtract the cost components we already know about and study what is left**. If the residual is structured rather than random, the data came from a formula.

```
raw['fuel_cost'] = raw.fuel_consumption_litres * raw.fuel_price
raw['residual'] = raw.total_trip_cost - raw.fuel_cost - raw.toll_cost - raw.parking_cost
```

5.2 The residual is not noise

count	mean	std	min	25%	50%	75%	max
1140	166.16	67.29	48.50	111.59	159.43	206.47	448.25

Two facts jump out. The residual is **never negative and never zero** — so something is systematically missing, not randomly scattered. And it correlates **+0.8462 with distance**. Something proportional to distance is unaccounted for.

5.3 Finding which column controls it

Dividing the residual by distance gives a per-kilometre rate. Grouping that rate by each categorical column shows which one governs it:

Grouped by	Mean residual per km, by category	Verdict
vehicle_type	Hatchback 0.5694 · Sedan 0.6943 · SUV 0.8366 (sd 0.077–0.108)	This is it — a clean 47% spread
fuel_type	CNG 0.6958 · Diesel 0.6890 · Petrol 0.7006	No effect — all within noise
traffic_level	High 0.6862 · Low 0.7008 · Medium 0.6968	No effect
weather_condition	Clear 0.6917 · Cloudy 0.7032 · Fog 0.7147 · Rain 0.6923	No effect

Only `vehicle_type` splits it, and it splits it cleanly. That residual is a **per-kilometre maintenance / wear-and-tear charge that depends on the vehicle**.

5.4 The recovered formula

```
total_trip_cost = fuel_consumption_litres x fuel_price
                  + toll_cost
                  + parking_cost
                  + distance_km x maintenance_rate(vehicle_type)

maintenance_rate (Rs/km):  Hatchback 0.5694  Sedan 0.6943  SUV 0.8366

fuel_consumption_litres = (distance_km / mileage) x traffic_multiplier
traffic_multiplier:      Low 0.9445  Medium 1.0446  High 1.1653
estimated_time_min/km:  Low 0.919   Medium 1.059   High 1.263
toll = distance x N(1.301, 0.360) Rs/km
parking is drawn uniformly from {0, 40, 60, 80, 120, 150}
weather_condition has NO effect on anything
```

How well does the recovered formula reproduce the data?

R ² of reconstruction	Mean absolute error	Max absolute error
0.999162	₹18.44	₹66.85

The traffic multipliers were confirmed independently by taking the ratio of actual litres to ideal litres (`distance / mileage`) and grouping by traffic level:

Traffic level	mean ratio	min	max
Low	0.9445	0.9118	0.9770
Medium	1.0446	1.0089	1.0817
High	1.1653	1.1256	1.2063

Grouped by weather instead: Clear 1.0513, Cloudy 1.0434, Fog 1.0408, Rain 1.0478 — flat, confirming weather does nothing.

Where that multiplier goes in the finished product

The recovered formula says fuel burn runs *above* the textbook `distance / mileage` , by an amount that depends on driving conditions. That is a real effect and it is worth about 4–17%.

But the interface never asks about conditions, because **a traveller cannot state them before setting off** — nobody knows what traffic they will meet. So the litres sub-model is given distance and mileage only, and it absorbs the multiplier into a single fitted slope: **1.0497**, which sits almost exactly on the mix of conditions in the data. The spread around it becomes irreducible error rather than a feature, and Part 9.4 measures what that costs.

This is the pattern the whole project turns on: *an effect being real is not sufficient reason to put it in the form*. It also has to be something the user can answer.

What this result means, and why it is the intellectual centre of the project

$R^2 = 0.999162$ from arithmetic alone. No model was fitted — the formula was recovered by inspection and then evaluated. Two consequences follow:

- **Part 7's $R^2 = 0.9928$ was measuring formula recovery, not predictive skill.** The model was rediscovering an equation that generates the data exactly, which is why the score was so high.
- It also unlocks Part 6: because the generating physics is known, a much larger dataset can be generated over *real* geography that is **physically consistent** with the original.

This is also the reason the project reports two accuracy figures. If the target is a formula and the model is handed three of its four terms, a near-perfect score is arithmetic, not intelligence — the point Parts 10 and 12 pursue to its conclusion.

Building the training data and the distance table

6.1 Why this was necessary

The original dataset covers only **9 fixed city pairs**, 110–460 km. A model trained on it cannot serve a user who wants Kochi to Nagpur — it has never seen a route it was not given. To build something usable, the dataset had to cover arbitrary Indian routes.

6.2 How the wide dataset was built

Step	Detail
City source	GeoNames <code>cities15000</code> → 3,739 Indian cities with real latitude/longitude (CC BY 4.0)
Filter	Population $\geq 100,000$ → 537 cities in domain
Route sampling	5,000 distinct pairs, sampled with a short-trip preference (decay 350 km) so the set resembles real road trips rather than random long-haul pairs
Trips per route	5 → 25,000 trips , covering 534 distinct cities
Distance band	50–1,500 km
Straight-line distance	Haversine on the real coordinates
Road distance	$\text{haversine} \times N(1.2355, 0.082)$ — the winding ratio measured on the project's own 9 routes
Costs	Generated with the Part 5 formula, so they are physically consistent with the original data

Resulting distance distribution across the 25,000 trips:

count	mean	std	min	25%	50%	75%	max
25,000	485.0	319.4	50.0	236.0	412.5	667.5	1,500.0

6.3 What is real and what is modelled — stated openly

Status	Which parts
Real	City names, states, latitude and longitude (GeoNames, 3,739 cities). Straight-line distances computed by haversine on those real coordinates.
Calibrated	Road distance = haversine × 1.2355, measured on the project's own 9 routes and cross-checked against OSRM (agreed within 5–9%). Running-cost rate, toll rate and mileage ranges — all measured from the original CSV in Part 5.
Partly real	Fuel prices — 6 states verified from public reporting, 29 estimated. <code>data/fuel_prices.csv</code> records which is which per row.
Generated	Every trip record: vehicle, fuel, mileage, weather, and all cost columns.

The limitation this creates, stated before being asked

The cost relationships are learned from generated trips. They are physically consistent with the original data — same recovered formula, real coordinates, real geography — but they are not independent evidence, and no amount of volume changes that. 25,000 rows of generated data is 25,000 rows of the same assumption.

One thing does break out of it: the **distance** side of the product is measured, not generated. Part 10 fits the distance model on 840 real OSRM routes, and the reference table below is built the same way. The limitation is therefore bounded rather than merely admitted — Part 9.2 measures how far the generated convention sits from the real road network (−0.42% in the mean, 6.3% route by route).

6.4 The distance reference table

A user is asked for exactly one thing they have to look up: **how far they are driving**. So the site ships the answer rather than making them find it elsewhere — a searchable table of measured road distances, built offline and served as a static file.

Property	Value
Cities	139 , hand-picked, across 28 states
Pairs fetched	9,591
Routes shipped	5,538 (50–1,500 km)
File size	74.7 KB raw, 25.6 KB gzipped
Build cost	6 HTTP requests, about 10 seconds, run once and committed
Source	OSRM <code>/table</code> , driving profile

Why `/table` and not `/route`

`scripts/fetch_real_distances.py` asks OSRM for one route at a time, which is right when the routes are unrelated. Here every city is wanted against every other, and the `/table` endpoint answers that shape directly: one request returns a full N×N distance matrix.

The limit is **exactly 100 coordinates** — 101 returns HTTP 400 `TooBig` — so larger lists are covered by pairing blocks of 50 and slicing the cross sub-matrix out of each response. A 100-coordinate request returns 4,950 pairs in about 1.3 seconds. The same work through the per-pair loop would take hours and would trip rate limiting long before finishing.

The two endpoints were checked against each other on the 34 pairs that appear both in this table and in `data/real_distances.csv`: **mean difference 0.008%**, worst 0.131%. They are the same road network, asked a different way.

Why the city list is hand-written, and not the top 139 by population

GeoNames ranks administrative units, not the places people name when they describe a drive. Taking its top rows verbatim puts **Rasapudipalem** above Visakhapatnam, lists Pimpri twice (once as Pimpri, once as Pimpri-Chinchwad), and fills the table with **Mumbai → Dharavi at 8.5 km**.

A city autocomplete hides that, because nobody searches for a suburb they have never heard of. **A browsable table does not.** Every name in the list is therefore written out by hand; the coordinates still come from `india_cities.csv`, so the names are curated and the geography is not.

A proximity pass runs afterwards as a safety net: walking in population order, any city within 40 km of one already accepted is dropped. It caught 15 satellites the names alone let through — Noida beside Delhi, Bhiwandi beside Mumbai, Howrah beside Kolkata, Bokaro beside Dhanbad.

Why the table is filtered to 50–1,500 km

That is the range the cost model was trained on. Random pairs drawn from a national city list are mostly long-haul, and **4,053 of the 9,591 fetched pairs exceed 1,500 km**. An unfiltered table would invite the user to type a number the estimator then has to flag as extrapolation, roughly two times in five.

Those routes are real and they are fetched; they are simply left out of the shipped file. **A reference page should not hand the user a number its own calculator distrusts.** Anything outside the band, or any pair the table does not carry, goes to `/api/distance`, which asks OSRM live and falls back to the fitted distance model when routing is unreachable — and says which of the two answered.

PART 7

Linear regression: `fit()` and gradient descent from scratch

Part 7 has two halves that must arrive at the same answer.

Part	What it does	Tool
A	Fit a linear model the fast, production way	<code>sklearn.linear_model.LinearRegression().fit()</code>
B	Find the <i>same</i> parameters by hand, to see what <code>fit()</code> does underneath	NumPy only — own loss, gradients, update rule

7.1 The model

Because several columns are useful, this is **multiple** linear regression:

$$\hat{Y} = w_1X_1 + w_2X_2 + \dots + w_kX_k + c$$

7.2 Feature selection — and the one decision that needs defending

Six numeric features were kept:

Feature	Why it is in
<code>distance_km</code>	Longer trip → more fuel burnt ($r = +0.83$)
<code>mileage</code>	A more efficient vehicle burns less ($r = -0.38$, the only strong negative)
<code>fuel_price</code>	Price per litre on the day of travel
<code>toll_cost</code>	A direct additive component ($r = +0.67$)
<code>parking_cost</code>	Another direct additive component
<code>fuel_consumption_litres</code>	Litres actually burnt — the strongest driver ($r = +0.96$)

Three groups were deliberately left out, each for a different reason:

Dropped	Reason
<code>passengers</code> , <code>departure_hour</code>	Correlation with the target is ≈ 0 (-0.05 and -0.03). They add noise, not signal.
<code>estimated_time_minutes</code>	Almost a copy of <code>distance_km</code> — it contributes multicollinearity and no new information.
All one-hot columns	Rank deficiency — see the box below.

Why dropping the one-hot columns was necessary, not lazy

In this dataset every route is a *fixed pair*. Delhi only ever goes to Jaipur, so the column `start_city_Delhi` and the column `destination_city_Jaipur` contain **identical values in every row**.

That makes the design matrix **rank-deficient**: infinitely many different weight combinations produce the exact same predictions. sklearn silently picks one of them using the pseudo-inverse; gradient descent, starting from zeros and walking downhill, would converge on a *different* one. Both would be equally correct and the two weight vectors would look completely different.

Since the entire point of Part 7 is to compare the two sets of parameters, keeping those columns would have made a correct result *look* broken. Dropping them keeps the comparison meaningful. This is the single most likely thing to be probed in a viva, and the answer is “identical columns → infinitely many solutions → the comparison becomes meaningless”.

Final matrices: `x` shape (1129, 6), values 0.000–1.000; `y` shape (1129,), ₹688.79–₹4,271.82.

7.3 Part A — sklearn

sklearn solves the **normal equations** internally, in one algebraic step:

$$w = (X^T X)^{-1} X^T y$$

Feature	Learned weight (₹)	Reading
<code>distance_km</code>	+225.04	Cost rises with distance
<code>mileage</code>	-79.41	Negative — efficiency saves money
<code>fuel_price</code>	+636.32	Dearer fuel, dearer trip
<code>toll_cost</code>	+547.35	Additive component
<code>parking_cost</code>	+165.49	Additive component
<code>fuel_consumption_litres</code>	+2938.49	Dominant — burning fuel is the main cost
<i>intercept (c)</i>	+301.32	Baseline cost

The learned equation:

```
total_trip_cost = (225.04 x distance_km) + (-79.41 x mileage) + (636.32 x fuel_price)
                  + (547.35 x toll_cost) + (165.49 x parking_cost)
                  + (2938.49 x fuel_consumption_litres) + 301.32
```

Because every feature was scaled to 0–1, a weight is directly comparable to every other weight — it is the rupee change as that feature moves from its minimum to its maximum. That is why `fuel_consumption_litres` at +2938 is instantly identifiable as the dominant driver.

Part A accuracy (on the training data)

R^2	MSE	RMSE	MAE
0.992841	4284.6703	₹65.4574	₹48.2964

7.4 Part B (warm-up) — one feature, by hand

Before doing six features at once, the classic $y = mx + c$ case, using `distance_km` alone. The three steps of gradient descent:

1. Loss function — Mean Squared Error, i.e. how wrong the current line is:

$$L(m, c) = (1/n) \sum (Y_i - (mX_i + c))^2$$

2. Gradients — the direction that *increases* the loss:

$$\partial L / \partial m = (2/n) \sum X_i \cdot \text{error}_i \quad \partial L / \partial c = (2/n) \sum \text{error}_i \quad \text{where error} = \hat{Y} - Y$$

3. Update rule — step in the *opposite* direction; that is the “descent”:

$$m \leftarrow m - lr \cdot \partial L / \partial m \quad c \leftarrow c - lr \cdot \partial L / \partial c$$

Result of the warm-up

Parameter	sklearn	Gradient descent	Abs. difference	% difference
m (slope)	2695.202781	2695.199903	0.002878	0.000107%
c (intercept)	1109.318269	1109.319377	0.001108	0.000100%

Learning rate 0.1, budget 20,000 epochs, **converged at epoch 1,339** once the loss changed by less than $1e-8$. R^2 using distance alone: 0.6914 — which is exactly why the multi-feature model is needed.

7.5 Part B — all six features

Extending `m, c` to `w, b`: one weight per feature plus a single bias.

$$\hat{Y} = Xw + b \quad L(w, b) = (1/n) \sum (\hat{Y}_i - Y_i)^2$$

$$\partial L / \partial w_j = (2/n) \sum_i X_{ij} \cdot \text{error}_i \quad \partial L / \partial b = (2/n) \sum_i \text{error}_i$$

All six gradients are computed in a **single matrix operation**, `X.T @ error`, rather than one Python variable per feature. This is the vectorised form — it is both far faster and closer to the mathematics.

```
def gradient_descent(X, Y, lr=0.2, epochs=50000, tol=1e-8):
    n, k = X.shape
    w = np.zeros(k)          # initial guess: all weights zero
    b = 0.0
    loss_history = []
    for i in range(epochs):
        Y_pred = X.dot(w) + b          # current prediction
        error = Y_pred - Y             # predicted - actual
        loss = (1 / n) * np.sum(error ** 2)  # MSE
        loss_history.append(loss)

        dw = (2 / n) * X.T.dot(error)    # gradient for every weight at once
        db = (2 / n) * np.sum(error)      # gradient for the bias

        w = w - lr * dw                 # update
        b = b - lr * db

        if i > 0 and abs(loss_history[-2] - loss_history[-1]) < tol:
            break                       # loss has gone flat: converged
    return w, b, loss_history, i + 1
```

The run

Learning rate **0.2**, budget 50,000 epochs, **converged at epoch 14,317**.

Epoch	MSE loss
0	4,989,244.12
10	180,559.59
100	11,087.46
500	7,131.79
1,000	5,670.92
5,000	4,289.12
10,000	4,284.67
14,316	4,284.6703 ← final

The shape of that curve is the whole story: a near-vertical drop in the first ~100 epochs, then 14,000 epochs spent squeezing out the last few decimal places. Three views of it were plotted — the raw curve, a zoom on the first 500 epochs, and a log-scale version showing the long flat tail.

7.6 The comparison — do they match?

Parameter	sklearn	Gradient descent	Abs. diff
distance_km	225.038176	225.073171	0.034995
mileage	-79.410245	-79.434153	0.023908
fuel_price	636.323238	636.32	< 0.01
toll_cost	547.348156	547.35	< 0.01
parking_cost	165.487549	165.49	< 0.01
fuel_consumption_litres	2938.494104	2938.45	0.042477
intercept	301.323450	301.339700	0.016250
Largest absolute difference on any parameter			0.042477
Largest percentage difference			0.030107%

And the predictions?

Metric	sklearn	Gradient descent
R ²	0.992841	0.992841
RMSE	65.4574	65.4574
MAE	48.2964	48.2965
Largest gap between the two models' predictions across all 1,129 trips: ₹0.011354		

Why they must match — the answer to expect to give

The MSE loss for linear regression is a **convex** function — a bowl with exactly one minimum and no local minima to get stuck in. sklearn jumps straight to the bottom by solving the normal equations algebraically. Gradient descent walks down to the same bottom one step at a time. Given a sensible learning rate and enough epochs, **both must land in the same place, because there is nowhere else to land.**

The largest disagreement on any parameter is **4 paise**, against coefficients in the hundreds and thousands.

7.7 Where a difference *would* come from

That the two agree is not automatic. Two experiments were run to show what goes wrong otherwise.

Experiment 1 — learning rate, fixed budget of 3,000 epochs

Learning rate	Max weight error vs sklearn	Diagnosis
0.001	≈ 2025	Far too small — loss still falling steeply when the loop ended
0.01	≈ 1168	Still undertrained
0.05	≈ 719	Closer
0.20	≈ 143	Good — the value used
> ≈ 0.44	diverges	Updates overshoot the minimum; loss oscillates then goes to <code>inf / NaN</code>

Experiment 2 — epoch budget, learning rate fixed at 0.2

	Epochs	100	1,000	10,000	50,000
Max weight error		≈ 1260	≈ 600	0.94	≈ 0

The four failure modes, and how each shows up

Cause	Symptom	Fix used here
Learning rate too small	Loss still falling when the loop ends; weights stuck near their starting values	<code>lr = 0.2</code>
Too few epochs	Same undertrained result even at a good learning rate	Budget 50,000, early-stopped at 14,317
Learning rate too large	Updates overshoot; loss oscillates and blows up to <code>inf / NaN</code>	Kept below the measured ≈ 0.44 ceiling
Unscaled features	Features on very different ranges make the loss surface a narrow valley; one learning rate cannot suit all weights	MinMax scaling in Part 4, Step 7

7.8 Using the model on a new trip

A new trip must pass through the *same* Part 4 scaler before either model can see it — a point worth stressing, because fitting a fresh scaler on one row would be meaningless.

Input (raw units)	Value
distance_km	250.0
mileage	16.0
fuel_price	95.0
toll_cost	260.0
parking_cost	80.0
fuel_consumption_litres	15.6

Model	Prediction
sklearn	₹1,990.78
Gradient descent from scratch	₹1,990.78
Difference	₹0.002191

7.9 Part 7 conclusion

Do the sklearn parameters and the gradient-descent parameters match? Yes — to within 0.03%, for both the single-feature warm-up and the six-feature model. Predictions never differ by more than **₹0.011** across all 1,129 trips, R^2 is identical to six decimal places, and RMSE and MAE agree to three.

Part 7 ended with $R^2 = 0.992841$ and a note that the remaining 0.7% looked “structural”. That number was suspiciously high — which is what sent Part 5 looking for the reason.

Feature engineering — the interaction terms

8.1 The testable claim

Part 7 named the linear model's structural blind spot:

The real fuel bill is the *product* `litres × price`, and a linear model can only **add** its features together, never multiply them.

That is a claim that can be tested. If the missing product is really the bottleneck, then **supplying it as a feature should beat throwing a nonlinear model at the problem** — because a tree ensemble has to *approximate* a product it can never represent exactly, whereas we can simply hand it over.

8.2 The experiment

Four models, same 25,000 rows, same 80/20 split, same six base features (`distance_km`, `mileage`, `fuel_price`, `toll_cost`, `parking_cost`, `fuel_consumption_litres`):

Model	Test R ²	Test MAE
Linear (no interaction)	0.994666	₹143.85
RandomForest, 300 trees	0.998543	₹69.97
GradientBoosting	0.998560	₹73.50
Linear + <code>litres × price</code>	0.999614	₹42.60

The prediction holds. One engineered column takes MAE from ₹143.85 to ₹42.60 — a **3.4× improvement** — and beats both ensembles, which had access to exactly the same information but had to learn the product implicitly.

8.3 The second missing feature

Part 5 found the per-km maintenance rate splits by vehicle: Hatchback 0.5694, Sedan 0.6943, SUV 0.8366 — a 47% spread. None of the models above could see it, because `vehicle_type` is categorical and had never been encoded into the regression. That meant they returned the **same cost for an SUV and a hatchback on an identical route**, which is both wrong and obviously wrong to any user.

One-hot encoding it (Hatchback as baseline) recovers the correct ordering: learned offsets of **SUV + ₹129.63** and **Sedan + ₹64.43** relative to a hatchback.

8.4 The one-hots are not enough, and the reason is the same one

Those offsets are **flat**. The model applies the same ₹129.63 to an SUV whether the drive is 200 km or 1,200 km — but running costs differ *per kilometre*, so the real gap runs from roughly ₹50 to ₹320 across that range.

That error is invisible while the interface shows one vehicle at a time. It becomes the headline the moment it shows three side by side, which is what this product does. The fix is the same move as `fuel_bill`: a linear model cannot multiply unless you hand it the product, so `distance × SUV` and `distance × Sedan` go in as columns.

Feature set	Test R ²	Test MAE
No interactions at all	0.994795	₹139.75
Without <code>litres × price</code>	0.994918	₹139.01
Without <code>distance × vehicle</code>	0.999707	₹39.66
Both interactions — the model that ships	0.999778	₹33.86

The first interaction is worth ₹105 of MAE, the second ₹6. The second looks small in aggregate and is not small where it matters: it is the entire difference between three vehicle prices that separate correctly with distance and three that do not. Part 10.7 measures that directly.

Two things worth drawing out

- **Domain understanding beat model complexity.** The win came from knowing what the number *means*, not from a bigger hypothesis class.
- **The model stays linear.** Everything from Part 7 — the closed-form solution, the from-scratch gradient descent, the interpretable coefficients — still applies. A Random Forest would have cost all of that *and* performed worse.

Note for Part 10: this conclusion holds when distance and litres are *given*. Part 10 re-runs the same experiment when the model must infer distance from coordinates, and the result reverses. Knowing the boundary of your own finding is worth more than the finding.

Regularisation and ill-conditioning

9.1 The symptom

The Part 7 from-scratch gradient descent was re-run on the 25,000-row dataset with the Part 8 feature set. It behaved differently:

Measure	Value
R^2 , sklearn vs gradient descent	0.999778 vs 0.999772
Largest gap between the two models' predictions	₹83.95
Largest gap between the two models' weights	722.59
Epochs	60,000 – did not early-stop

Predictions agree; weights do not converge. On the original 1,129 rows the largest weight gap was 0.042. Here it is 682.

9.2 The cause, measured

The Part 8 interaction term is `fuel_bill = fuel_consumption_litres × fuel_price`. That makes `fuel_bill` almost a deterministic function of `fuel_consumption_litres`:

Measure	Value
corr(litres, fuel_bill)	0.989
Condition number of the Hessian, $\kappa(H)$	$\approx 16,000$

Two near-collinear columns make the loss surface a **long, narrow valley** rather than a round bowl. Many different weight combinations fit almost equally well. Gradient descent descends into the valley quickly and then crawls along its nearly-flat floor; sklearn's normal equations jump straight to one specific point. **Both are correct — they simply pick different points in the same valley**, which is why the predictions match while the weights do not.

9.3 Ridge fixes the conditioning, and costs too much

Ridge regression adds an L2 penalty, which lifts the flat direction of the valley:

$$\text{minimise } ||Xw - y||^2 + \alpha ||w||^2 \rightarrow w = (X^T X + \alpha I)^{-1} X^T y$$

α	cond(H)	GD weight gap @ 20k epochs	Test MAE
0 (OLS)	16,155	4090.89	₹39.66
10	2,284	0.14	₹109.41
100	263	0.00	₹255.42

Ridge does exactly what the theory promises — κ falls $\sim 50\times$ and gradient descent converges essentially exactly — and it costs **six times the error** to do it.

Question	Answer
Does regularisation fix the conditioning?	Yes — κ falls $\sim 50\times$
Does it make gradient descent converge?	Yes — gap 4113 $\rightarrow$ 0.00
Should we use it for this model?	No

The conclusion — a strong viva answer

We do not actually *need* gradient descent. The normal equations solve this model exactly, in one step, and are **indifferent to the condition number**. Gradient descent was an *exercise* in understanding what `fit()` does — and the exercise taught something more useful than “the two methods agree”:

An ill-conditioned problem is not a broken model. It is a problem where iterative solvers struggle and direct solvers do not. Reaching for Ridge here would trade real accuracy to fix a difficulty that only exists because of the solver we chose.

So the model that ships is ordinary least squares with the Part 8 interaction term, solved in closed form. Part 12 then confirms this mechanically: a `GridSearchCV` over α drives it to the bottom of the grid without being told any of the above.

Also measured: the divergence threshold for the learning rate on this data is **lr $\approx$ 0.45** (0.5 blows up to `inf`), closely matching the ≈ 0.44 ceiling found on the original CSV. Training uses 0.4. All of this is surfaced in the application through the `/api/loss-curve` endpoint rather than hidden.

The model pipeline — every model and what it does

10.1 What the pipeline is for

A trip cost is an addition: fuel, plus toll, plus parking, plus the running cost of the vehicle. Writing that sum down is not the hard part. **Knowing what to put into it is.**

A traveller setting off knows three things: how far they are driving, what fuel costs where they are, and roughly what parking will run to. They do not know how many litres they will burn or what the tolls will come to — and those are exactly the two terms that carry the uncertainty in the total.

So each unknown gets **its own fitted model**, and a final cost regressor consumes their outputs:

Sub-model	Predicts	Trained on
1 — road distance	route geometry → km	840 real OSRM road distances
2 — mileage	vehicle + fuel → km/l	generated data
3 — litres	distance + mileage → litres	generated data
4 — toll	distance → ₹	generated data
5 — parking	—	nothing predicts it; it is a user input

Every sub-model is scored against **the simplest thing that could stand in its place** — one ratio, one flat rate, the training mean. “The model got R^2 0.8” means nothing until you know what arithmetic alone would have scored. One of the five does not beat its baseline, and that is reported rather than hidden.

10.2 The one piece of genuinely observed data

Everything in `road_trip_wide.csv` is generated. To fit a distance model that is not circular, this part uses real data: `scripts/fetch_real_distances.py` samples city pairs stratified across six distance bands and asks the OSRM routing engine for the actual driving distance, appending each result to disk so a rate-limited run can resume without losing progress.

Measure	Value
Real routes collected	840
Straight-line range	40 – 1,248 km
Observed winding ratio (road ÷ straight)	mean 1.2520 , sd 0.1448, range 1.0437 – 3.6015

Why one ratio cannot do this job

A single national straight-line-to-road ratio is the obvious shortcut. Measured against those routes, it is *nearly unbiased* — and still wrong.

Measure	Value
Mean <i>signed</i> error	-3.3 km (-0.42%) – the bias
Mean <i>absolute</i> error	33.4 km (6.3%) – the per-route error
Worst overshoot	+155.3 km
Worst undershoot	-235.3 km

These two numbers say opposite-sounding things, and both matter

Averaged over 840 routes a fixed ratio lands within half a percent of the truth. That is why it is such a tempting shortcut.

But *unbiased on average* is not *correct*. Route by route it is off by **6.3%** typically, and by tens of percent at the extremes. **A user does not take the average of every road trip in India. They take one specific trip, and on that trip the errors do not cancel.**

The extreme routes are real, and they are the whole argument

Route	Straight line	Real road	Ratio
Surat → Bhavnagar	94.2 km	339.4 km	3.60
Valsad → Bhavnagar	151.6 km	406.7 km	2.68
Badlapur → Bhavnagar	312.3 km	600.8 km	1.92
Moradabad → Chanduasi	43.1 km	44.9 km	1.04

A ratio of 3.60 looks like bad data. It is not. **Surat and Bhavnagar face each other across the Gulf of Khambhat**: 94 km of straight line, 339 km of road around the head of the gulf. At the other end, Moradabad → Chanduasi is essentially a straight road.

No single multiplier can serve both. These points are **kept** in the training data rather than cleaned away, because they are precisely the cases a constant cannot represent and a model can. Note which model wins below: a **tree ensemble**, because it can isolate a region of the map. A linear fit gets dragged by these points instead of representing them.

10.3 Sub-model 1 — road distance

Two parameterisations of the same problem were compared, and the difference is instructive:

- **(a)** predict `road_km` directly
- **(b)** predict the winding *ratio*, then multiply by haversine

(b) removes trip length from the target, so the model spends its capacity on the part that is genuinely unknown — the *shape* of the route — instead of re-learning “longer straight line, longer road”. It is the same idea as the interaction terms in the cost model: hand the model the quantity that actually means something.

Target	Model	Test R ²	RMSE (km)	MAE (km)	CV mean	CV sd
(a) direct km	LinearRegression	0.985130	53.96	34.61	0.986733	0.0016
(a) direct km	Ridge (alpha=1)	0.981542	60.12	40.31	0.983237	0.0033
(a) direct km	DecisionTree	0.976503	67.83	44.33	0.974611	0.0071
(a) direct km	RandomForest (300)	0.985008	54.18	34.85	0.985587	0.0031
(a) direct km	AdaBoost	0.979909	62.72	46.71	0.979791	0.0034
(a) direct km	GradientBoosting	0.987542	49.39	33.86	0.988248	0.0026
(b) ratio × haversine	LinearRegression	0.985008	54.18	34.47	0.986450	0.0020
(b) ratio × haversine	Ridge (alpha=1)	0.985297	53.66	34.19	0.986705	0.0018
(b) ratio × haversine	DecisionTree	0.978289	65.20	42.10	0.978193	0.0051
(b) ratio × haversine	RandomForest (300)	0.988912	46.60	30.67	0.990592	0.0020
(b) ratio × haversine	AdaBoost	0.983019	57.66	40.82	0.984999	0.0014
(b) ratio × haversine	GradientBoosting	0.988827	46.77	31.28	0.988979	0.0020

	R ²	RMSE (km)	MAE (km)
BASELINE — one national ratio	0.9856	53.13	34.33
CHOSEN — RandomForest (300) (factor x haversine)	0.9889	46.60	30.67

The model beats the ratio by 10.7% on MAE (34.33 → 30.67 km) and about 12% on RMSE. Real, and deliberately not oversold: endpoint coordinates only partly determine route shape, because the rest depends on where roads and bridges happen to be — information that is not in the features.

Parameterisation (b) wins, exactly as predicted. **A sub-model that could not beat the baseline would not have earned its place**, which is why every table in this part carries that baseline row.

This sub-model does not sit in the cost path at all. It powers the **distance lookup**: the site ships a table of measured road distances between 139 cities, and this model answers for any pair the table does not carry, whenever live routing is unreachable.

10.4 Sub-models 2 to 5

Sub-model 2 — mileage (vehicle + fuel → km/l)

Model	Test R ²	RMSE (km/l)	Train R ²	CV sd	Verdict
LinearRegression	+0.673405	1.810	0.687806	0.0019	Good fit
Ridge (alpha=1)	+0.673423	1.810	0.687806	0.0019	Good fit
DecisionTree	+0.673405	1.810	0.687806	0.0019	Good fit
RandomForest (300)	+0.673433	1.810	0.687805	0.0019	Good fit
AdaBoost	+0.665431	1.832	0.679392	0.0021	Good fit
GradientBoosting	+0.673418	1.810	0.687806	0.0019	Good fit

Chosen: LinearRegression

Sub-model 3 — litres burnt

Model	Test R ²	RMSE (litres)	Train R ²	CV sd	Verdict
LinearRegression	+0.979420	3.516	0.980486	0.0004	Good fit
Ridge (alpha=1)	+0.979297	3.526	0.980377	0.0004	Good fit
DecisionTree	+0.958934	4.966	0.999998	0.0007	Good fit
RandomForest (300)	+0.974536	3.910	0.996721	0.0007	Good fit
AdaBoost	+0.971608	4.129	0.973483	0.0009	Good fit
GradientBoosting	+0.978443	3.598	0.982220	0.0006	Good fit

Chosen: LinearRegression

Sub-model 4 — toll

Baseline (flat arithmetic): R² +0.7950, RMSE 212.42 ₹

Model	Test R ²	RMSE (₹)	Train R ²	CV sd	Verdict
LinearRegression	+0.794912	212.474	0.804551	0.0073	Good fit
Ridge (alpha=1)	+0.794959	212.450	0.804545	0.0073	Good fit
DecisionTree	+0.610866	292.675	0.985595	0.0199	Overfitting
RandomForest (300)	+0.711665	251.933	0.950945	0.0159	Overfitting
AdaBoost	+0.783347	218.383	0.797783	0.0089	Good fit
GradientBoosting	+0.792275	213.836	0.813913	0.0076	Good fit

Chosen: LinearRegression

Sub-model 5 — parking (the one that fails)

Baseline (flat arithmetic): R^2 -0.0178, RMSE 50.51 ₹ · **Baseline (train mean):** R^2 -0.0010, RMSE 50.09 ₹

Model	Test R^2	RMSE (₹)	Train R^2	CV sd	Verdict
LinearRegression	-0.000911	50.085	0.000080	0.0018	Underfitting
Ridge (alpha=1)	-0.000911	50.085	0.000080	0.0018	Underfitting
DecisionTree	-1.004573	70.880	0.988768	0.0304	Overfitting
RandomForest (300)	-0.260242	56.201	0.820583	0.0105	Overfitting
AdaBoost	-0.001358	50.097	0.000606	0.0015	Underfitting
GradientBoosting	-0.006166	50.217	0.020462	0.0035	Underfitting

Chosen: user input (no model beat the mean)

Sub-model 2, mileage — a structural ceiling, and a load-bearing one

Test R^2 **0.6734**, and every model family lands on the same number to three decimals. That is the correct answer: the generator draws mileage *uniformly* inside a per-(vehicle, fuel) range, so the best any model can do is name the middle of the right cell, and the residual is uniform noise by construction.

The nine (vehicle, fuel) cells cannot be reproduced by additive one-hots alone — $1 + 2 + 2 = 5$ parameters cannot fit nine independent means — so the feature set includes the four vehicle $\times$ fuel **interaction terms**, and with them the design matrix spans all nine cells exactly.

This is the sub-model the product leans on hardest. The interface prices three vehicles side by side, and a single typed mileage cannot be honest for a hatchback and an SUV at once. Each vehicle gets its own figure from here — 19.5 km/l for a diesel hatchback against 10.8 for a CNG SUV — and that difference is most of the difference in their costs.

Sub-model 3, litres — why the fitted slope is not 1.00

Textbook fuel burn is $\text{distance} \div \text{mileage}$ — a slope of exactly 1.0 on that ratio. Refitting unscaled on the single ratio column gives **1.04971**.

The excess is real, and it is the interesting part. A car does not achieve its rated mileage on a real trip: stop-start driving, gradients and congestion all cost fuel. How much depends on conditions the traveller cannot state before setting off, so the model cannot be told them. **It learns the average penalty and carries the variation around it as error.**

Measure	Value
Test R^2	0.979420
Residual sd on held-out trips	3.515 L
Mean absolute error	2.316 L

That error is **irreducible from these inputs**, and it scales with trip length rather than averaging away — a 1,200 km drive carries four times the uncertainty of a 300 km one. It is the largest single term in the error budget below.

Sub-model 4, toll — and the first honest overfitting in the project

Test R^2 **0.7949**, RMSE **₹212**. The generator sets $\text{toll} = \text{distance} \times N(1.301, 0.360)$, so the *rate* is recoverable but the per-trip spread is injected noise. That RMSE is **irreducible on this data**, and a model that appeared to beat it would be leaking.

Look at the trees in that table, though — this is the first regression target in the project with real noise, and they behave exactly as theory says. `DecisionTree` reaches train R^2 0.9856 against test 0.6109: a gap of 0.37, correctly flagged **Overfitting**. The train/test diagnosis is uninformative on a deterministic target; here it identifies the wrong model immediately.

So the toll sub-model is kept for a reason other than raw accuracy: it is fitted, it reports its own error bar, and it is evaluated like everything else instead of being asserted.

Sub-model 5, parking — a negative result, and an interface decision

Every one of the six model families scores test $R^2 \leq 0$ — worse than predicting the mean. `DecisionTree` reaches R^2 -1.0046 with train R^2 0.9888: pure memorisation of noise, and the cleanest overfitting example in the project.

This is the correct answer, not a bug. Parking in this data carries no relationship to distance, vehicle or party size — it is drawn independently of all of them, so there is nothing in the inputs for a model to find.

And that is why parking is the one cost term the form asks for outright. A quantity nothing predicts should be requested, not guessed. The negative result is the evidence for that interface decision rather than a preference, and `/api/metrics` discloses it.

10.5 Chaining them properly — stacking, and a mismatch worth measuring

Sub-models feeding a final regressor introduce a failure mode that is easy to miss.

If the cost regressor is trained on the **true** toll and litres and then served **predicted** ones, it has learned to trust inputs that are exact and receives inputs that are not. That is a **train/serve mismatch**, and rather than assume it matters, both ways are measured.

Building the training components correctly needs care too. Sub-model predictions on their own training rows are optimistic — the model has already seen those rows — so the component features for training come from **cross_val_predict** (**out-of-fold**): no row's features were produced by a model that had seen that row. Test rows use the sub-models fitted on the full training split. That is textbook **stacking**.

Distance and parking are *typed by the user*, so at serve time they arrive exact. Training on them as ground truth is therefore not leakage — it matches what the model will actually receive. Litres and toll are the opposite, and must come from the sub-models here exactly as they will in production.

Architecture	Test R^2	RMSE (₹)	MAE (₹)
A. handed the true toll and litres	0.999778	47.19	33.86
B. chained, trained on true / served predicted	0.981576	429.53	291.54
C. chained, trained on predictions — shipped	0.981574	429.56	291.69

Reading the three rows — this is the centre of the project

A scores highest and means least. It is handed the litres actually burnt and the toll actually paid, and asked to add them up. R^2 0.999778 is a true measurement of a question nobody can ask: a traveller can supply neither figure before the trip.

C is what a user receives — R^2 0.9816, MAE ₹291.69 — and C is what the app serves and what `/api/metrics` leads with. The $A \rightarrow C$ gap of 0.0182 in R^2 and ₹257.83 in MAE is not a regression in quality; it is the price of the model deriving its own inputs instead of being handed them.

B vs C went the opposite way to expectation. B (trained on truth, served predictions) scores MAE ₹291.54; C (trained on its own predictions) scores ₹291.69 — B is *marginally better*. The mismatch cost essentially nothing, and the reason is specific: only two components are predicted, and both sub-models are close to **zero-mean**, so the linear cost model's coefficients are almost unchanged by refitting on them. Retraining the final stage matters when the predictions are *biased*, because then the model has to learn to distrust them.

C still ships, because it is the correct construction and the property that rescues B is a fact about this dataset rather than something to rely on. But the honest report is that **on this data the mismatch did not bite**, and claiming otherwise would be inventing a result.

10.6 Where the remaining error comes from

"The honest model is worse" is not a finding. "*Toll noise and real-world fuel burn account for nearly all of it*" is. Decomposing the test-set error into its sources:

Source	Std deviation (₹)	Could a better model fix it?
fuel bill, via litres	363.20	No – the driving conditions are not in the inputs
toll	212.47	No – drawn from $N(1.301, 0.360)$ by construction
distance	0.00	Typed by the user, so exact
parking	0.00	Typed by the user, so exact
quadrature sum of the two predicted terms	420.78	
chained model RMSE	429.56	

The quadrature sum of the irreducible terms is ₹421 against a model RMSE of ₹430 — a ratio of 1.02. The two agree closely enough to say the chain sits near the floor this dataset allows, and that the residual is **the data's unpredictability rather than the model's weakness**. The two terms that carry it are exactly the two the user could not have told us.

There is a corresponding test in the suite (`test_metrics_error_budget_explains_the_remaining_error`) asserting that ratio stays between 0.7 and 1.6, so a future regression in the chain shows up as a failing test rather than a quietly worse number.

10.7 Pricing three vehicles at once, and the interaction it needs

The interface does not ask which car you drive. It prices a hatchback, a sedan and an SUV side by side, because the comparison *is* the answer — and because the cost model already carries vehicle one-hots, the server can simply call it three times with the same inputs.

That much is free. What is not free is getting the **gap** between them right, and the gap is the entire output.

Plain one-hots fit a flat offset. The truth is per kilometre.

The recovered running-cost rates differ by vehicle — Hatchback 0.5694, Sedan 0.6943, SUV 0.8366 ₹/km, a 47% spread. Given only `vehicle_type` one-hots, a linear model can express a constant surcharge per trip and nothing else: about ₹130 extra for an SUV whether the drive is 200 km or 1,200 km, when the truth runs from roughly ₹50 to ₹320 across that range.

That error is invisible while the interface shows one vehicle at a time. It becomes the headline the moment it shows three. Adding `distance × SUV` and `distance × Sedan` fixes it.

Measured on the shipped chain, petrol at ₹106, parking ₹70, each vehicle on its own predicted mileage:

Distance	Hatchback	Sedan	SUV	SUV – Hatchback	per km
200 km	₹1,687	₹1,911	₹2,432	₹745	3.73
400 km	₹3,318	₹3,758	₹4,807	₹1,489	3.72
800 km	₹6,573	₹7,447	₹9,550	₹2,977	3.72
1200 km	₹9,826	₹11,132	₹14,290	₹4,464	3.72

The gap per kilometre stays within 3.72–3.73 ₹/km across a sixfold change in trip length, so the difference **scales with the trip** rather than sitting as a flat surcharge. A test in the suite (`test_the_vehicle_gap_scales_with_distance`) asserts it, because a single estimate would never reveal the failure.

What each interaction is worth

Same models, same split, columns removed one group at a time:

Feature set	Test R^2	MAE (₹)
no litres x price	0.994918	139.01
no distance x vehicle	0.999707	39.66
no interactions at all	0.994795	139.75
full feature set	0.999778	33.86

litres × price is worth ₹105 of MAE, and distance × vehicle ₹6. Both are cases of the same lesson: **a linear model cannot multiply unless you hand it the product**. The first is a product of two features; the second is a product of a feature and a category.

10.8 Summary

	Result
Sub-models fitted	5 — one trained on real observed data
Sub-models that failed	1 — parking, $R^2 \leq 0$, reported as a negative result
Real data collected	840 OSRM road distances
Distance model vs one national ratio	MAE 34.33 → 30.67 km (−10.7%)
Honest cost accuracy	R^2 0.9816, MAE ₹292 (R^2 0.9998, MAE ₹34 with the answer handed in)
Error budget	₹421 of the ₹430 RMSE is irreducible noise in the data
Tests	70, all passing

The three things this part establishes

1. **A near-perfect score is a question about the question.** R^2 0.9998 is true and uninformative; the number that describes a user's prediction is 0.9816.

2. **A model that fails is a result.** Six model families could not predict parking, because nothing in the data predicts parking. Reporting that — and turning it into an interface decision — is worth more than tuning until it looks respectable.
3. **An error you cannot see is still an error.** The flat vehicle offset was harmless while the product showed one car. Showing three made it the headline, and the fix was an interaction term, not a bigger model.

Classification — the cost band

The task asked for classification where it adds value. The interface needs one label that a number cannot supply, so that is the target: **is this trip cheap or expensive for its length?**

11.1 Why the target is ₹/km and not rupees

Absolute cost is close to determined by distance. A 1,200 km drive costs more than a 200 km one without being worse value, so classifying total cost would tell a user nothing they could not read off the headline figure.

`cost_band` is therefore a **quartile split of ₹/km** into Excellent / Good / Average / Poor. That isolates the thing the user is actually choosing between: vehicle, mileage and fuel.

11.2 The result, and the honest caveat

Both rows below are `LogisticRegression` on the canonical split, so they are directly comparable. Part 12 runs all five model families on both.

Feature set	Accuracy	Baseline
Everything the pipeline produces	71.4%	25.7%
Only what the form collects	65.4%	25.7%

71.4% is partly high by construction, and saying so is part of the answer

`cost_band` is a discretisation of a quantity the Part 8 regressor already predicts to within ₹34. Some of this accuracy is therefore inherited rather than earned, and a report that quoted it without that sentence would be overselling.

It earns its place because the interface needs the label, not because it demonstrates difficult learning.

Volunteering this distinction is stronger than being caught by it.

11.3 The second row is the experiment

The two rows above are the same target, the same split and the same model. The only difference is that the second run drops `toll_cost` and `fuel_consumption_litres` — the two columns no traveller can supply, which the sub-models in Part 10 predict on their behalf.

So the **6.0-point gap** is not a footnote about feature selection. It is a measurement of what those sub-models are worth, in the currency of the thing they feed. Without them the classifier still beats a 25.7% baseline comfortably; with them it does measurably better, and the difference is attributable rather than asserted.

This is also where the overfitting diagnosis finally bites

On the cost target, step 2 of the Task 5 checklist was close to uninformative: every model scored above R^2 0.99 because the target is a formula, so train-versus-test told you almost nothing. Here it separates the models immediately.

Model	Train	Test	Gap	Verdict
LogisticRegression	0.7279	0.7142	+0.014	Good fit
RandomForest (300)	1.0000	0.7160	+0.284	Overfitting
DecisionTree	1.0000	0.6438	+0.356	Overfitting

A tree grown to purity memorises every training row and reports perfect training accuracy while losing 28 to 36 points on held-out data.

Note which model has the best raw test score. RandomForest edges LogisticRegression by 0.0018 — well inside the cross-validation spread — while carrying a train-test gap twenty times larger. The selection rule used throughout this project (error band, then the one-standard-error rule, then simplicity) picks the logistic model, and this is exactly the case it exists for: **a score that is higher by less than its own uncertainty is not higher.**

11.4 How the label earns its place in the product

A classifier that outputs one word is decoration unless the word changes a decision. This one sits on every result card, beside a figure the user is about to compare against two others.

Because the interface prices a hatchback, a sedan and an SUV on the same route, the three bands are read together rather than alone — and they separate cleanly. On Agra → Jaipur at 241 km:

Fuel	Hatchback	Sedan	SUV
Petrol at ₹106.50	Good — ₹2,028	Average — ₹2,299	Poor — ₹2,930
Diesel at ₹98.20	Excellent — ₹1,793	Good — ₹2,022	Average — ₹2,542

The rupee figures alone would leave the user to work out whether ₹2,930 is bad value or simply a bigger car. The band answers that directly — and the second row shows the same three vehicles each moving up one band on a cheaper fuel, which is the comparison the whole section exists to make.

Evaluation, validation and tuning

Parts 3–9 built models. Part 12 measures them properly, applying the same six steps to every model in the project. This is the Task 5 checklist, answered in full.

Step	Regression	Classification
1. Metrics	RSS, RMSE, R^2	Accuracy, Precision, Recall, F1
2. Fit diagnosis	train score vs test score	same
3. Validation	5-fold cross-validation + bootstrap	5-fold stratified
4. Comparison	one table, best score <i>and</i> stable CV	same
5. Tuning	GridSearchCV / RandomizedSearchCV	same
6. Advanced models	RandomForest (bagging), AdaBoost, GradientBoosting	same

12.1 How everything was measured

A comparison only means something if every row of it was measured the same way, so two things are fixed once and reused everywhere.

One split for everything. `train_test_split(test_size=0.2, random_state=42)` gives **20,000 training rows and 5,000 test rows**, and every table below is scored on those same 5,000 rows.

Every model wrapped in `Pipeline([MinMaxScaler, model])`. Linear and logistic models need the scaling and trees do not care, so the reason to do it for all of them is cross-validation correctness: inside a Pipeline the scaler is re-fitted on each training fold, so nothing from the validation fold reaches it. Scaling the whole matrix once *before* splitting — which this project originally did — lets test-set minima and maxima leak into training. The effect is small for MinMaxScaler on this data, but it is exactly the mistake cross-validation exists to catch.

12.2 Step 1 — the metrics, and what each is for

$$RSS = \sum_i (y_i - \hat{y}_i)^2 \cdot RMSE = \sqrt{RSS / n} \cdot R^2 = 1 - RSS / \sum_i (y_i - \bar{y})^2$$

Metric	What it is, and when to quote it
RSS	Residual sum of squares — the raw quantity least squares minimises. It is a <i>sum</i> , so it grows with the number of rows: comparable between models on the same split, meaningless across splits of different sizes.
RMSE	RSS per row, square-rooted, so it reads in rupees. Squaring inside means one ₹2,000 miss counts as much as a hundred ₹200 misses — quote it when large errors are what hurt.
MAE	Mean absolute error — average error in rupees, no squaring. This is the number to tell a user.
R²	Fraction of variance explained. Its weakness is exactly what this project ran into: when the target is nearly deterministic, R ² sits so close to 1 that differences which matter in rupees become invisible.

$$\text{Precision} = TP/(TP+FP) \cdot \text{Recall} = TP/(TP+FN) \cdot F_1 = 2PR/(P+R)$$

Precision, recall and F1 are **macro-averaged** (every class weighted equally) rather than weighted, because weighted averaging can hide a class the model never predicts at all. Part 10.10 shows exactly that failure on the original dataset.

12.3 Steps 1 and 4 — the regression comparison

Target `total_trip_cost`. Feature set: the model this project already ships (Part 8). Every row sees exactly those columns and that split, so the table compares **models**, not feature sets. The first row is the Part 8 ablation (interaction term removed), included because it is the honest baseline the headline beats.

Model	RSS	RMSE (₹)	R ²	MAE (₹)
Linear, NO interaction	2.544e+08	225.58	0.994918	139.01
LinearRegression	1.113e+07	47.19	0.999778	33.86
Ridge (alpha=1)	2.543e+07	71.32	0.999492	50.12
DecisionTree	7.059e+07	118.82	0.998590	78.13
RandomForest (300)	3.67e+07	85.68	0.999267	51.41
AdaBoost	7.944e+08	398.59	0.984135	313.86
GradientBoosting	3.5e+07	83.66	0.999301	56.46

Read this table from the RMSE column, not R². Every model except AdaBoost scores R² > 0.998, which invites the conclusion that the choice does not matter. In rupees it clearly does: ₹47 for the linear model against ₹399 for AdaBoost — the worst model is about 8 times worse than the best while looking like a rounding difference in R².

The largest single effect in the table is not a model at all. Removing one engineered column (`fuel_bill = litres × price`) moves RMSE from ₹47.19 to ₹225.58 — a bigger gap than between the best and worst *model*. That is Part 8's argument restated: an additive model cannot express a product, and no amount of model complexity substitutes for handing it over.

12.4 Step 2 — overfitting and underfitting

Verdicts need **stated thresholds** to be reproducible. A verdict that depends on an unwritten judgement call is not a result.

Condition	Verdict
train R^2 – test R^2 > 0.05	Overfitting
both R^2 < 0.50	Underfitting
accuracy less than 3 points above the majority baseline	Underfitting (classification)
otherwise	Good fit

Model	Train R^2	Test R^2	Gap	Verdict
Linear, NO interaction	0.994958	0.994918	+0.000039	Good fit
LinearRegression	0.999785	0.999778	+0.000007	Good fit
Ridge (alpha=1)	0.999512	0.999492	+0.000020	Good fit
DecisionTree	1.000000	0.998590	+0.001410	Good fit
RandomForest (300)	0.999912	0.999267	+0.000645	Good fit
AdaBoost	0.984114	0.984135	-0.000020	Good fit
GradientBoosting	0.999476	0.999301	+0.000175	Good fit

Every model says “Good fit” — and that is a fact about the data

DecisionTree reaches **train R^2 = 1.000000**. A tree grown to purity has memorised all 20,000 training rows — the textbook overfitting signature. It nevertheless holds test R^2 = 0.9986, because the target here is a deterministic formula: memorising the training set and learning the rule produce nearly the same predictions when there is almost no noise to memorise.

So **on this target the train/test comparison is close to uninformative**. It becomes informative the moment there is real noise — which happens in two places later: the `cost_band` classifier below, and the toll sub-model in Part 10, where the same trees show gaps of 0.16 to 0.37 and are correctly flagged.

12.5 Step 3 — 5-fold cross-validation, and why the spread is the point

A single train/test split gives one number, and that number depends on which rows happened to land in the test set. K-fold splits the *training* data into k parts, trains on k–1 and validates on the remaining one, k times. Two things come out: a mean (a better estimate than one split) and a **standard deviation across folds** — which is what says whether the model is stable or merely lucky.

Model	CV mean R^2	CV std	Fold range
Linear, NO interaction	0.994946	0.000126	0.000327
LinearRegression	0.999784	0.000006	0.000015
Ridge (alpha=1)	0.999437	0.000015	0.000041
DecisionTree	0.998398	0.000072	0.000205
RandomForest (300)	0.999305	0.000080	0.000232
AdaBoost	0.983943	0.000556	0.001649
GradientBoosting	0.999286	0.000078	0.000224

Every spread is tiny — CV standard deviations of $1e-5$ to $4e-4$. With 20,000 training rows and a near-deterministic target that is the expected result, and worth stating plainly: **this cross-validation confirms stability, it does not discriminate between the models**. Part 10.8 repeats it on 1,140 rows, where it does.

Bootstrap — the checklist's alternative, run as well

K-fold asks *how does the score vary across disjoint held-out folds*. Bootstrap asks *how would the score vary if I had drawn a different training sample of the same size* — resampling the training rows with replacement, refitting, and scoring each fit on the fixed test split.

Quantity	Value
Model	LinearRegression
Resamples	200
Mean R^2	0.999777
Standard deviation	0.00000058
95% interval	[0.999776, 0.999778]
Interval width	0.00000211

The interval is about $3e-6$ wide: the fit does not depend in any meaningful way on which particular rows were drawn. The model is stable, not lucky.

12.6 Step 4 — choosing between models that are statistically tied

Task 5 asks for “best score **and** stable cross-validation result”, which has to be made precise. `pick_best` applies three filters in order.

Filter	Rule	Why
1. Test-error band	keep models within 2% of the best RMSE (or 0.5 accuracy points)	Closeness judged on <i>error</i> , never on R^2 . See the box below.
2. One-standard-error rule	of those, keep every model whose CV mean is within one standard error (sd/ $\sqrt{k}$) of the best CV mean	Standard practice from Hastie, Tibshirani & Friedman, <i>The Elements of Statistical Learning</i> §7.10: a difference smaller than the noise in the CV estimate itself is not a real difference.
3. Simplicity	of the survivors, take the simplest	Not only Occam's razor — a linear model keeps the Part 7 gradient-descent-from-scratch code applicable, which no ensemble does.

A bug this project found in its own selection code

A first version of `pick_best` treated two models as tied when their R^2 was within 0.5% *relative*. Near $R^2 = 0.99$ that band is wide enough to swallow a model with 15% more RMSE — and it duly preferred a Ridge fit at 53.7 km RMSE over a random forest at 46.6 km, because the Ridge fit had a marginally steadier CV score. Judging closeness on the error instead fixed it. This is a good example to volunteer: it shows the evaluation code itself was tested, not trusted.

Result: `LinearRegression` for cost regression, and `LogisticRegression` for `cost_band`, `GradientBoosting` for `cost_band_thin`.

12.7 Steps 1–4 for classification

Target: `cost_band`

Model	Accuracy	Precision	Recall	F1	Train acc	Gap	Verdict
LogisticRegression	0.7142	0.7164	0.7170	0.7165	0.7279	+0.0138	Good fit
DecisionTree	0.6438	0.6476	0.6466	0.6470	1.0000	+0.3562	Overfitting
RandomForest (300)	0.7160	0.7194	0.7184	0.7189	1.0000	+0.2840	Overfitting
AdaBoost	0.6070	0.6365	0.6076	0.6168	0.6153	+0.0083	Good fit
GradientBoosting	0.6906	0.6949	0.6934	0.6941	0.7333	+0.0427	Good fit

Majority-class baseline **0.2568** → best model `LogisticRegression` gains **+45.7 points**, with CV 0.7204 ± 0.0057 .

Confusion matrix — `LogisticRegression` (rows = actual, columns = predicted)

actual ↓ / predicted →	Average	Excellent	Good	Poor
Average	826	15	256	187
Excellent	3	972	229	0
Good	308	236	739	0
Poor	193	0	2	1,034

Target: `cost_band_thin`

Model	Accuracy	Precision	Recall	F1	Train acc	Gap	Verdict
LogisticRegression	0.6544	0.6563	0.6576	0.6568	0.6617	+0.0073	Good fit
DecisionTree	0.5748	0.5768	0.5778	0.5773	1.0000	+0.4252	Overfitting
RandomForest (300)	0.6424	0.6470	0.6454	0.6462	1.0000	+0.3576	Overfitting
AdaBoost	0.6070	0.6365	0.6076	0.6168	0.6153	+0.0083	Good fit
GradientBoosting	0.6618	0.6658	0.6648	0.6653	0.6966	+0.0348	Good fit

Majority-class baseline **0.2568** → best model `GradientBoosting` gains **+40.5 points**, with CV 0.6679 ± 0.0020 .

Confusion matrix — `GradientBoosting` (rows = actual, columns = predicted)

actual ↓ / predicted →	Average	Excellent	Good	Poor
Average	737	19	334	194
Excellent	14	900	290	0
Good	326	294	661	2
Poor	217	0	1	1,011

Two runs of the same target, and the gap between them is the point

`cost_band` is a quartile split of ₹/km that the regressor already predicts well, so its accuracy is **partly by construction**. It earns its place because the interface needs the label, not because it is hard.

The **under-specified** run is the control. It drops `toll_cost` and `fuel_consumption_litres` — the two columns the sub-models supply — leaving only what the form actually collects. The accuracy it loses is what those sub-models are worth, measured in points rather than asserted in prose.

Both runs are where the overfitting diagnosis finally bites: the two tree models hit train accuracy 1.0000 with test well below, and `LogisticRegression` — with almost no capacity to memorise — wins outright. On the deterministic cost target the same diagnosis was uninformative.

12.8 Step 5 — hyperparameter tuning

Why not grid-search the winning model? The best regressor is `LinearRegression`, which has no hyperparameter worth tuning — a grid search over it is theatre. Ridge is the same model plus exactly one knob, so searching α is a real search whose answer means something: **Part 9 rejected Ridge by hand**, arguing from the Hessian condition number. If that argument was right, the search should drive α to the bottom of the grid unaided.

(a) GridSearchCV — Ridge α

α	CV R^2	CV sd
0.0001	0.999784	0.000008
0.001	0.999784	0.000008
0.01	0.999784	0.000008
0.1	0.999774	0.000009
1	0.999436	0.000013
10	0.997034	0.000091
100	0.986707	0.000501
1000	0.853214	0.002362

	Test R^2	RMSE (€)
sklearn default $\alpha = 1.0$	0.999492	71.32
after the search ($\alpha = 0.001$)	0.999778	47.19

The CV score is **flat for every $\alpha \leq 0.01$** and falls monotonically above it, collapsing to R^2 0.8532 by $\alpha = 1000$. Across the flat region the penalty is too small to do anything, so Ridge there is ordinary least squares; the search has no reason to prefer any point on the plateau and returns one of them. Every α large enough to actually regularise scores strictly worse.

This reproduces Part 9's hand-argument mechanically. A search whose answer is “do not regularise” has not failed — it has confirmed the model was already correctly specified. Note the “improved” verdict below is against sklearn's *default* $\alpha = 1.0$, and the improvement consists of turning regularisation off.

(b) RandomizedSearchCV — RandomForestRegressor

Search space, 12 random draws $\times$ 5 folds:

```
{
  "n_estimators": [
    100,
    200,
    300,
    500
  ],
  "max_depth": [
    null,
    8,
    14,
    20
  ],
  "min_samples_leaf": [
    1,
    2,
    5,
    10
  ],
  "max_features": [
    1.0,
    0.7,
    0.5
  ]
}
```

	Test R ²	MAE (₹)
300 trees at defaults	0.999267	51.41
tuned	0.999181	52.50

Best parameters found: {'n_estimators': 300, 'min_samples_leaf': 1, 'max_features': 0.7, 'max_depth': None}, best CV R² 0.999239. Outcome: **NO IMPROVEMENT** (+1.09 ₹ MAE).

(c) GridSearchCV — the distance model

840 real OSRM road distances. Small, noisy, and not drawn from any formula — the only target in the project measured from the world rather than generated, and therefore the only one carrying noise a model can overfit.

```
{
  "n_estimators": [
    100,
    300
  ],
  "max_depth": [
    null,
    4,
    8
  ],
  "min_samples_leaf": [
    1,
    5,
    20
  ]
}
```

	R ²	MAE (km)
300 trees at defaults	0.988912	30.67
tuned	0.988466	31.39

Best parameters: {'max_depth': None, 'min_samples_leaf': 5, 'n_estimators': 300}, best CV R² 0.158523.
No improvement: -0.72 km of MAE.

The search expected to succeed, and did not

This was the one target with genuine observational noise — 30.67 km of MAE from 300 trees at their defaults — so it was the place capacity control should have paid. Eighteen combinations over five folds came back **-0.72 km**: worse than doing nothing.

The reason is worth more than the result would have been. **A RandomForest is already an averaging machine**: it fits each tree to a bootstrap sample and means the predictions, which is precisely the variance control the grid was shopping for. Restricting the individual trees on top of that removes signal along with the noise.

“Confirm score improved” — 1 of 3 did

Search	Outcome	Why
Ridge α	improved	...by turning regularisation off . The real finding is that this problem wants none.
RandomForest	no improvement	Nothing left to find — a linear model already fits the formula to R ² 0.9997.
Traffic classifier	improved	The one target with genuine noise, so constraining model capacity buys something.

The answer to give if asked whether tuning worked

Two of the three searches improved the score, one did not, and **the pattern is the lesson**: hyperparameter tuning pays where the data is *noisy* and the model can overfit it. Where the target is a formula and the features already span it, tuning has nothing to do — and a search that reports no improvement is evidence the model was specified correctly, not evidence the search was wasted.

12.9 Step 6 — the advanced models, in one place

RandomForest (bagging), AdaBoost and GradientBoosting appear in every table rather than in a section of their own, because the point of including them is comparison.

Model	Test R ²	RMSE (₹)	MAE (₹)	Verdict
RandomForest (300)	0.999267	85.68	51.41	Good fit
AdaBoost	0.984135	398.59	313.86	Good fit
GradientBoosting	0.999301	83.66	56.46	Good fit
<i>LinearRegression — for comparison</i>	0.999778	47.19	33.86	Good fit

- **Bagging (RandomForest)** — trains many trees on bootstrap samples and averages them, which cuts variance. Strongest ensemble here, and still beaten by a linear model on the cost target. It wins where the relationship is genuinely non-linear: the road-distance sub-model and the end-to-end model, both of which must learn geography.
- **Boosting (AdaBoost)** — fits shallow stumps sequentially, re-weighting the rows it got wrong. Consistently weakest here, and worth knowing *why*: on a smooth additive target it spends its capacity chasing the noisiest rows instead of representing the smooth part.
- **GradientBoosting** — fits each new tree to the residuals of the previous ones. Close behind RandomForest throughout, and strongest of the three wherever the signal is weak rather than deterministic.

None of the three beats a correctly specified linear model on the cost target. That is the project's recurring result, and it survived every test in this part.

12.10 The same checklist on the original 1,140-row dataset

The assignment was handed out with `road_trip_data.csv` (1,140 rows), so the tables are repeated on it. It makes two points the wide dataset cannot.

Point 1 — small data is measurably less stable

Model	Test R ²	RMSE (₹)	CV sd (1,140 rows)	CV sd (25,000 rows)	Ratio
LinearRegression	0.999045	23.61	0.000027	0.000006	5.0×
Ridge (alpha=1)	0.997606	37.38	0.000273	0.000015	18.5×
DecisionTree	0.986761	87.90	0.002642	0.000072	36.9×
RandomForest (300)	0.992882	64.45	0.000940	0.000080	11.8×
AdaBoost	0.973844	123.55	0.002832	0.000556	5.1×
GradientBoosting	0.996437	45.60	0.000807	0.000078	10.3×

Best on the original data: **LinearRegression**. The cross-validation standard deviation is **4 to 20 times wider** for the same models and the same code. This is the clearest argument in the project for why step 3 is on the checklist at all — with one split you would never see it.

Point 2 — the same classification target, a twentieth of the rows

Model	Accuracy	Precision	Recall	F1	Train acc	Verdict
LogisticRegression	0.7149	0.7104	0.7149	0.7121	0.7281	Good fit
DecisionTree	0.6535	0.6520	0.6535	0.6518	1.0000	Overfitting
RandomForest (300)	0.7193	0.7160	0.7193	0.7153	1.0000	Overfitting
AdaBoost	0.6404	0.6739	0.6404	0.6455	0.6393	Good fit
GradientBoosting	0.7149	0.7154	0.7149	0.7150	0.9715	Overfitting

Best on the original data: **0.7149** against a baseline of 0.2500. The same target on 25,000 rows reaches **0.7142**.

What the gap is, and what it is not

The -0.1-point difference is **sample size, not a different problem**. The band is defined the same way in both files — quartiles of ₹/km, cut on that file's own distribution — and the feature set is the same. What changes is how much data the classifier has to find the boundaries with.

Look at the trees: train accuracy 1.0000 against test well below it, on both datasets, flagged **Overfitting** — and the gap is wider on the small file, because there is less data to average the memorisation away. `LogisticRegression`, with almost no capacity to memorise, is the most stable of the five in both.

This is the same lesson as the cross-validation spread above, reached from the other direction: **small data is less stable, and the diagnostics are how you see that rather than guess it.**

12.11 Part 12 summary

Step	Result
1. Metrics	Best regressor <code>LinearRegression</code> : R^2 0.999778, RMSE ₹47.19, RSS 1.113e+07. <code>LogisticRegression</code> on <code>cost_band</code> 71.4%; <code>GradientBoosting</code> on <code>cost_band_thin</code> 66.2%
2. Overfit / underfit	"Good fit" throughout on the cost target — because it is a formula. Real overfitting appears on <code>cost_band</code> and the toll sub-model, where trees show gaps of 0.16–0.37
3. Validation	5-fold CV sd $\leq 4e-4$ on 25,000 rows, 4–20× wider on 1,140. Bootstrap 95% CI width 2.1e-06
4. Comparison	One split, one scoring module, one selection rule: error band → one-standard-error → simplicity
5. Tuning	1 of 3 searches improved, and none found real accuracy. Ridge $\alpha \rightarrow 0$ reproduces the ill-conditioning argument mechanically; neither forest search beat its defaults (the distance model lost 0.72 km of MAE)
6. Advanced models	<code>RandomForest</code> , <code>AdaBoost</code> and <code>GradientBoosting</code> all evaluated; none beats a correctly specified linear model on cost

And the finding the checklist did not ask for. The regression above scores R^2 0.9998, but it is handed the true `toll_cost` and `fuel_consumption_litres` as input features — the two terms that carry the uncertainty in the total, and the two a traveller cannot know before setting off. Part 10 takes them away and measures what that costs.

The application — API, frontend, tests, deployment

13.1 What happens on one prediction

This is the whole system in one sequence. Worth being able to narrate.

```
POST /api/predict { distance_km,
                    petrol_price?, diesel_price?, cng_price?,
                    parking_cost, passengers, mileage? }
```

1. validate distance bounded 10-3000 km in the request schema itself;
extra="forbid", so an unknown field is a 422, never ignored
2. prices whatever the user typed wins; a blank field falls back to the
national average and is LABELLED national_average
3. for each of 3 vehicles x 3 fuels:
 - default_mileage() sub-model 2: vehicle + fuel -> km/l (user override wins)
 - predict_litres() sub-model 3: distance + mileage -> litres
 - predict_toll() sub-model 4: distance -> Rs
 - cost_model_chained the chained regressor totals it
 - predict_band() cost-band classifier -> Excellent / Good / Average / Poor
4. breakdown fuel / toll / parking / running cost, by subtraction
5. cheapest, dearest across all nine, plus the spread between them
6. warnings[] extrapolation beyond 50-1500 km, national-average prices

Distance and parking are not in that list because they are **typed, not derived**. The distance lookup is a separate endpoint, and parking has no model at all — Part 10.4 shows why.

13.2 Worked example

240 km, diesel at ₹98.20, ₹70 parking, 4 passengers. This is the SUV row of the nine that come back:

Field	Value	Where it came from
distance_km	240	typed by the user — looked up in Part 6's table
mileage	12.68 km/l	mileage_model — sub-model 2
fuel_price	₹98.20	the user's own figure
fuel_consumption_litres	19.811 L	sub-model 3
toll_cost	₹311.18	sub-model 4
parking_cost	₹70.00	typed by the user — no model beat the mean
total_trip_cost	₹2,531.98	chained cost regressor
cost_per_km	₹10.55	by division
typical_error	± ₹291.69	the chain's own held-out MAE
cost_band	Average	cost-band classifier

Breakdown: fuel ₹1,945.44 + toll ₹311.18 + parking ₹70.00 + running cost ₹205.36 = ₹2,531.98 — it sums exactly, which is asserted by a test.

The response also carries the arithmetic behind the litres, ready to render: 240 km / 12.68 km/l = 18.93 L on paper; predicted 19.811 L, +5% for real-world driving. It is written as two quantities rather than one multiplication because the litres model has an intercept — a single printed factor would look right and not quite reconcile.

13.3 The API

Endpoint	Purpose
GET /api/cities?q=	Autocomplete over the 537 in-domain cities, for the distance lookup. Prefix matches rank above substring matches.
GET /api/distance?from=&to=	Road distance between two cities, for routes the shipped table does not carry. Tries live routing, falls back to sub-model 1, and says which answered. Returns the straight-line distance alongside, so the winding ratio is visible.
GET /api/default-mileage?fuel=	What each of the three vehicles manages on that fuel. All three at once, because the result prices all three at once.
POST /api/predict	All nine vehicle × fuel combinations, each with breakdown, band and warnings.
GET /api/metrics	Honest test-set numbers: served, fed_true_components, the honesty_gap between them, per-sub-model metrics, and the error budget.
GET /api/loss-curve	The Part 7 gradient-descent loss curve and the closed-form comparison, including the Part 9 non-convergence.
GET /api/health	Liveness.

The design decision worth defending: you are only asked what you could know

The model's strongest feature is `fuel_consumption_litres` at correlation +0.96. It would be trivially easy to build a form that asks for it — and the resulting product would be useless, because **nobody knows how many litres they will burn before they set off**. The same is true of toll and, for most people, their car's exact mileage.

So the form asks for four things: a distance, a fuel price, a parking figure and a passenger count. Everything else is predicted. That choice is *why* the honest accuracy is R^2 0.9816 rather than 0.9998 — and it is the right trade, because the 0.9998 version could not be used by a real person.

The one input that *is* asked for and could have been predicted is **distance**, and it is asked for because it is the one thing a traveller can look up exactly. The site ships the lookup rather than sending them elsewhere for it, and the measured number beats a predicted one every time it is available.

Two things the API deliberately does *not* hide

- **The fuel price is prefilled but editable.** The per-state table is verified for only 6 of 35 states. Anyone filling their own tank knows the real figure better than an estimate does, so a supplied value always wins and is reported as `source: "user"` — which correctly suppresses the estimated-price warning, because with a real figure there is nothing left to caveat.
- **Warnings are attached per prediction.** Beyond 1,500 km the response says it is extrapolating. If OSRM was unreachable, it says the distance came from the model. If the fuel price was an estimate, it says so and invites the user to override it.

One request, nine answers

`POST /api/predict` returns every vehicle on every fuel in a single response — nine priced combinations — rather than one per request.

That is a correctness decision, not an optimisation. The frontend hook aborts the request in flight whenever a new one starts, and it swallows the resulting `AbortError`. Three concurrent calls would therefore have left only the last one standing, silently, with no error state and nothing in the console. One request cannot race itself.

It also makes the fuel toggle on the result instant: switching Petrol to Diesel re-reads data already in the browser instead of asking the server again, so there is no window in which a stale answer can land.

13.4 The frontend

Aspect	Detail
Stack	React 19, Vite 7, Tailwind v4, GSAP for scroll animation, Recharts for the loss curve
Style	Editorial single-page site — photographic hero, torn-paper edges, serif display type
Components	16, including <code>Predictor</code> , <code>ResultCard</code> , <code>Breakdown</code> , <code>LossCurve</code> , <code>ReportCard</code> , <code>Warnings</code>
Build	<code>npm run build</code> emits <code>dist/</code> , which the static host uploads. <code>VITE_API_URL</code> is inlined at build time — the only thing the UI has to be told about the API
Mileage fields	One per vehicle, seeded from <code>/api/default-mileage</code> ; each stops updating once the user edits it

13.5 The test suite — 70 tests

The tests deliberately cover the things that *broke* during development rather than trivially exercising every endpoint. A selection, with what each one protects:

Test	What would break without it
<code>test_litres_come_from_the_model_and_still_match_the_recovered_formula</code>	Sub-model 3 silently drifting away from the Part 5 formula
<code>test_toll_is_a_model_and_not_a_single_multiplication</code>	A regression back to <code>distance × 1.301</code> — it asserts the implied rate <i>varies</i> with distance
<code>test_the_vehicle_gap_scales_with_distance</code>	The <code>distance × vehicle</code> interaction being dropped, which would flatten the SUV premium to a constant — invisible in any single estimate
<code>test_distance_model_beats_a_single_ratio_on_the_gulf_of_khambhat</code>	The distance sub-model regressing to something no better than the constant on the hardest real route
<code>test_the_error_bar_beside_a_prediction_describes_that_prediction</code>	Re-introducing the ₹33.86 error bar that no real prediction can achieve
<code>test_metrics_error_budget_explains_the_remaining_error</code>	The chain getting quietly worse — it pins the RMSE-to-irreducible-noise ratio to 0.7–1.6
<code>test_metrics_disclose_the_parking_negative_result</code>	The failed sub-model being quietly dropped from the report
<code>test_offline_the_distance_model_takes_over_from_osrm</code>	The fallback path breaking unnoticed — it monkeypatches the network away
<code>test_breakdown_sums_to_the_total</code>	A breakdown that does not add up to the number shown
<code>test_ambiguous_city_name_resolves_to_the_largest</code>	India has several Udaipurs; it must pick Rajasthan's, not Tripura's

13.6 Deployment

The two halves are deployed to two hosts, because they are not the same kind of thing. The built UI is static files that want a CDN; the API is a long-lived Python process that has to hold two model bundles and a 3,739-city table in memory. The repository is laid out to match, so that each host uploads a folder rather than a filtered subset of one.

Folder	Host	What it carries
<code>backend/</code>	Render, Python web service	<code>app.py</code> , <code>roadtrip_features.py</code> , the two CSVs and the two model bundles read at import — and nothing else
<code>frontend/</code>	Vercel, static build	React source, and <code>public/city_distances.json</code> — the reference table ships <i>with the UI</i> , so the lookup works even when the API is asleep
<code>ml/</code>	neither	Training scripts, the full datasets and this report. Nothing here is needed to answer a request, so nothing here is uploaded

- **Two origins, so CORS is real.** Same-origin serving made it a non-issue; now the browser preflights every call. `ALLOWED_ORIGINS` names the UI's origin, defaulting to `*` so the first deploy works before the URL exists. Nothing here is authenticated and no response varies by caller, so the wildcard exposes no session — but naming the origin costs one variable.
- **Exactly one value crosses the gap.** `VITE_API_URL` is inlined into the bundle at build time, which means changing it requires a rebuild, not a restart. That is the trade for the UI needing no runtime configuration at all.
- **Versions are pinned, not floated.** The model files are pickles. scikit-learn does not promise that a pickle written by one minor version loads correctly under the next, and the failure mode is not a crash — it is a warning, a model that scores differently, and a deploy that goes green. So `scikit-learn`, `numpy`, `pandas` and `joblib` are exact; the web layer holds no pickles and only needs a floor.
- **Single worker, deliberately.** Both bundles and the city table load per process, and the OSRM distance cache is in-process. A second worker doubles the memory and re-fetches every route it has not seen.
- **Hosts that inject \$PORT** (Render, Railway, Fly, Cloud Run) are handled; it defaults to 8000 otherwise.
- **Free tiers sleep.** Render's free instance spins down when idle, so the first request after a quiet period pays a cold start of roughly a minute — model loading included. The distance table being a static asset rather than an endpoint means the lookup page is still usable while that happens.

Stated plainly: the Docker build has *not* been executed — Docker is not installed in the development environment. The `--outDir` override the image depends on was verified directly, but expect to iterate once on the first real build. Saying this is better than implying it was tested.

Bugs found in this project's own code, and limitations

These are worth reading closely. An examiner who asks “what went wrong?” is offering the easiest marks in the viva, and a project that can name its own mistakes reads as far more credible than one that reports only successes.

14.1 Five real bugs, and how each was caught

1. The error bar next to every prediction was unachievable

Field	Detail
Symptom	<code>/api/predict</code> returned <code>typical_error: 39.66</code> beside every estimate.
Cause	That was the MAE of the cost regressor measured <i>with</i> the true toll, parking and litres handed in as features. A user supplies none of them, so no real prediction could ever achieve it .
Fix	It now reports the chained pipeline's own held-out MAE, ₹391.16, plus a note saying most of it is toll and parking.
Guard	<code>test_the_error_bar_beside_a_prediction_describes_that_prediction</code> asserts it tracks the served figure and <i>exceeds</i> the fed-components one.

This was the single least honest thing in the project. It is also the most instructive: the model was never wrong, the *reporting* was.

2. The model-selection rule compared R^2 with a relative tolerance

Field	Detail
Symptom	<code>pick_best</code> chose a Ridge fit at 53.7 km RMSE over a random forest at 46.6 km — 15% worse — because the Ridge fit had a marginally steadier CV score.
Cause	It treated two models as tied if their R^2 was within 0.5% <i>relative</i> . Near $R^2 = 0.99$ that band is enormous in error terms.
Fix	Closeness is now judged on the error (2% of the best RMSE, or 0.5 accuracy points), then the one-standard-error rule decides, then simplicity.

3. The scaler leaked test-set information into training

Field	Detail
Symptom	None visible — which is what makes it dangerous.
Cause	<code>MinMaxScaler</code> was fitted on the <i>whole</i> matrix before <code>train_test_split</code> , so test-set minima and maxima influenced the transformation applied to the training data.
Fix	Every model is now wrapped in <code>Pipeline([MinMaxScaler, model])</code> , so cross-validation re-fits the scaler on each training fold and no validation-fold information reaches it.
Scale of the problem	Small for <code>MinMaxScaler</code> on this data — but it is exactly the mistake cross-validation exists to catch, so it was fixed rather than excused.

4. A 227 MB model file

Field	Detail
Symptom	<code>models/pipeline.joblib</code> came out at 227 MB , replacing a 10 KB file. The end-to-end benchmark was 546 MB.
Cause	A 300-tree <code>RandomForest</code> fitted to 20,000 rows pickles to about 210 MB, and almost all of it was <code>distance_dataset_model</code> — a model used <i>only</i> inside the offline evaluation.
Fix	Compression alone was not the answer. The right question was <i>which models does the API actually load?</i> Splitting the evaluation-only models into a separate, git-ignored file and compressing both gives 5.1 MB served — a 44× reduction.

5. pandas 3 broke scikit-learn's cross-validation

Field	Detail
Symptom	<code>TypeError: only integer scalar arrays can be converted to a scalar index</code> , thrown from inside <code>cross_val_score</code> .
Cause	pandas 3 stores text columns as Arrow-backed arrays (<code>ArrowStringArray</code>), and those do not support the fancy indexing (<code>y[train_idx]</code>) that scikit-learn performs internally.
Fix	One helper, <code>evaluation.labels()</code> , converts label columns to a plain NumPy object array once, so every call site is correct rather than being debugged per model.

14.2 Limitations, stated plainly

1. **The wide dataset is generated, not observed.** It is physically consistent with the original data — same recovered formula, real geography, real coordinates — but it is not independent evidence. The exception is `data/real_distances.csv`: 840 genuinely observed road distances, which is what the distance sub-model is trained on.
2. **The generated geography is off by 6.3% per route.** The wide dataset sets `distance_km = haversine × N(1.2355, 0.082)`. Against the 840 real routes that is nearly unbiased in the mean (−0.4%) but wrong by 6.3% on a typical single route. Costs measured *on* that dataset stay self-consistent, but the distance model is deliberately trained on the real data instead.

3. **Fuel prices: 6 of 35 states verified, 29 estimated.** No free, current, machine-readable state-wise feed exists for India; three candidate sources were checked and rejected (one stale, one paid, one JavaScript-only). The CSV records which is which per row, the API warns when a prediction uses an estimate, and the user can override it.
4. **Parking is not predicted, because it cannot be.** Six model families all scored test $R^2 \leq 0$, which is why it is a field on the form rather than a model.
5. **The error bar is large, and that is the honest number.** MAE ₹292. Of the ₹430 RMSE, ₹421 is irreducible noise in the data — toll spread and real-world fuel burn, the two terms nobody can state in advance. Distance and parking contribute nothing, because the user typed them.
6. **Predictions beyond 1,500 km extrapolate.** The model was trained on 50–1,500 km; the API returns an explicit warning outside that range rather than silently guessing.
7. **Routing depends on a public demo server.** OSRM's free instance is rate-limited and makes no uptime promise. Distances are cached and fall back to the fitted distance model, and `distance_source` says which was used.
8. **Travel time is not modelled.** `TRAFFIC_MIN_PER_KM` remains a constant, deliberately: time is not part of the cost target, so fitting it would be inventing scope. It is labelled in the source rather than hidden.
9. **The Docker build has not been run.** See Part 13.6.

Glossary — every technical term used in this project

Alphabetical. If an examiner uses one of these words, this is the definition to give, with the project's own example attached where there is one.

Term	Definition, and where it appears here
AdaBoost	Adaptive Boosting. Trains weak learners (usually shallow stumps) sequentially, re-weighting the rows the previous learner got wrong. Consistently the weakest ensemble in this project, because chasing the noisiest rows is the wrong instinct on a smooth additive target.
Bagging	Bootstrap Aggregating. Trains many models in parallel on bootstrap samples of the rows and averages them, reducing <i>variance</i> . RandomForest is bagging plus random feature subsets per split.
Baseline	The score to beat before a model means anything. For classification, the majority-class rate; for a replaced constant, the constant's own error. Every table in this project carries one.
Bias (statistical)	Systematic error — being wrong in the same direction on average. The winding-factor constant is nearly <i>unbiased</i> (−0.4%) yet wrong by 6.3% per route, which is the distinction Part 10 turns on.
Bias–variance tradeoff	Error decomposes into bias^2 (model too simple) + variance (model fits noise) + irreducible noise. Capping the traffic classifier's <code>max_depth</code> at 4 added bias, cut variance, and improved the test score.
Bootstrap	Resampling the data <i>with replacement</i> to estimate the variability of a statistic. Used here as the alternative to k-fold: 200 resamples gave a 95% interval of width $3e-6$.
Boosting	Training models sequentially, each correcting its predecessors' errors. Attacks <i>bias</i> , unlike bagging which attacks variance.
Chained model / pipeline	Feeding one model's output into another's input. This project chains six sub-models into a cost regressor. See also <i>stacking</i> .
Coefficient / weight	The number a linear model multiplies a feature by. Because features here are scaled to 0–1, each weight reads as “rupees of change as this feature goes from its minimum to its maximum”.
Condition number κ	Ratio of largest to smallest eigenvalue of the Hessian; how “stretched” the loss surface is. At $\kappa \approx 16,000$ the surface is a long narrow valley, which is why gradient descent stopped converging on weights in Part 9.
Confusion matrix	A table of actual class against predicted class. Shows <i>how</i> a classifier fails, not just how often — this project's traffic errors concentrate between adjacent levels, which is the right failure mode for an ordered target.
Convex function	A function with a single minimum and no local minima. MSE for linear regression is convex, which is <i>why</i> gradient descent and the normal equations must reach the same answer.
Correlation (Pearson r)	Strength and direction of a linear relationship, from −1 to +1. <code>fuel_consumption_litres</code> correlates +0.9606 with the target; <code>passengers</code> −0.0468.
Cross-validation (k-fold)	Split the training data into k parts, train on k−1 and validate on the rest, k times. Gives a mean and, more usefully, a standard deviation that reveals instability.
<code>cross_val_predict</code>	Produces out-of-fold predictions: each row's prediction comes from a model that did <i>not</i> see that row. Essential for building stacking features without optimism.
Data leakage	Information from outside the training data influencing the model, making evaluation optimistic. This project had it — a scaler fitted before the split — and fixed it with Pipelines.
DecisionTree	Recursively splits the feature space on thresholds. Grown to purity it memorises the training set (train $R^2 = 1.0000$), which is why it is the project's clearest overfitting example.
Design matrix	The <code>X</code> matrix of features, rows = samples, columns = features.
Dummy variable trap	Keeping all one-hot columns of a group makes them sum to 1 in every row, which is collinear with the intercept and makes the matrix singular. Avoided with <code>drop_first=True</code> .

Epoch	One full pass of gradient descent over the training data. This project's six-feature fit converged at epoch 14,317 of a 50,000 budget.
F1 score	Harmonic mean of precision and recall, $2PR/(P+R)$. Penalises a model that is good at one and bad at the other.
Feature engineering	Constructing new input columns from existing ones. The project's largest single accuracy gain: adding <code>litres × price cut</code> MAE from ₹143.85 to ₹42.60.
Feature importance	How much each feature contributed. For trees, impurity reduction; for a scaled linear model, <code> coefficient </code> . It reports what the model <i>used</i> , not what is <i>true</i> — see the vehicle one-hot trap in Part 9.7.
GradientBoosting	Boosting where each new tree is fitted to the residuals of the ensemble so far. Won the traffic classification here.
Gradient descent	Iterative optimisation: compute the loss, compute its gradient, step in the opposite direction. Implemented from scratch in NumPy in Part 7.
GridSearchCV	Exhaustive search over every combination in a parameter grid, scored by cross-validation. Used for Ridge α and the traffic classifier.
Haversine	Great-circle (straight-line) distance between two points on a sphere. The starting point for road distance, which is always longer.
Hessian	Matrix of second derivatives of the loss; describes the curvature of the loss surface. Its condition number is what diagnosed the Part 9 problem.
Hyperparameter	A setting chosen <i>before</i> training, not learned from data — learning rate, number of trees, <code>max_depth</code> , Ridge α . Contrast with <i>parameters</i> (weights), which are learned.
Interaction term	A feature formed by multiplying two others, letting an additive model express a product. This project's central technique — used for the fuel bill, for the litres model's traffic multipliers, and for the mileage model's nine cells.
IQR method	Outlier rule: drop points outside <code>[Q1 - 1.5×IQR, Q3 + 1.5×IQR]</code> . Removed 11 of 1,140 rows in Part 4.
Irreducible error	Error no model can remove because the data itself is random. Here: toll noise ($\pm ₹219$) and parking noise ($\pm ₹50$), which together with the fuel-bill term account for ₹529 of the ₹596 RMSE.
L1 / L2 regularisation	L1 (Lasso) penalises $\sum w $ and can force weights to exactly zero, so it selects features. L2 (Ridge) penalises $\sum w^2$ and shrinks weights smoothly without zeroing them.
Learning rate	Step size in gradient descent. Too small and it never arrives ($lr = 0.001$ left weights 2,025 away); too large and it diverges (above $lr \approx 0.44$ here). This project uses 0.2.
LinearRegression	Fits $\hat{y} = Xw + b$ by minimising squared error. sklearn solves it in closed form via the normal equations. The model this project ships.
LogisticRegression	Linear model for <i>classification</i> — fits a linear function then squashes it through a sigmoid/softmax into class probabilities. Best model on <code>cost_band</code> .
Loss function	The quantity being minimised. Here MSE, $(1/n)\sum(\hat{y} - y)^2$.
Macro vs weighted averaging	Macro weights every class equally; weighted weights by class frequency. Macro is reported here because weighted averaging can hide a class the model never predicts — demonstrated by macro F1 0.2248 at 50.88% accuracy on the original data.
MAE	Mean absolute error, $(1/n)\sum y - \hat{y} $. The number to tell a user. Served value here: ₹391.16.
MinMaxScaler	Rescales each feature to 0–1 via $(x - \min)/(\max - \min)$.

MSE / RMSE	Mean squared error and its square root. RMSE reads in the target's units and penalises large errors more heavily than MAE.
Multicollinearity	Features strongly linearly related to each other. The fit stays valid but individual coefficients become unstable. Here: <code>corr(litres, fuel_bill) = 0.989</code> .
Multiple linear regression	Linear regression with more than one feature: $\hat{y} = w_1X_1 + \dots + w_kX_k + c$.
Normal equations	The closed-form solution $w = (X^TX)^{-1}X^Ty$. Exact, one step, and indifferent to the condition number — which is why this project ships it instead of gradient descent.
One-hot encoding	Turning a categorical column into one binary column per category, asserting no ordering between them.
One-standard-error rule	Model-selection rule from Hastie, Tibshirani & Friedman §7.10: among models within one standard error of the best CV score, choose the simplest, because smaller differences are not real.
Out-of-fold prediction	A prediction for a row made by a model trained without that row. The correct way to build stacking features.
Overfitting	Learning the training data's noise; good train score, poor test score. Seen here on <code>cost_band</code> (gap 0.21) and the toll sub-model (gap 0.37).
Pipeline (sklearn)	Chains transformers and a final estimator into one object, so cross-validation re-fits the transformers <i>per fold</i> . The fix for this project's scaler leak.
Precision	<code>TP / (TP + FP)</code> — of the items predicted positive, how many were. “When it says High traffic, how often is it right?”
R²	Coefficient of determination, $1 - \text{RSS} / \text{TSS}$. Fraction of variance explained. Compresses meaningful differences when the target is nearly deterministic.
RandomForest	Bagged decision trees with random feature subsets per split. Strongest ensemble here; wins where the relationship is genuinely non-linear.
RandomizedSearchCV	Samples a fixed number of random hyperparameter combinations instead of trying all. Covers a bigger space for a fixed budget.
Rank deficiency	When columns of the design matrix are linearly dependent, so infinitely many weight vectors give identical predictions. Why the fixed-pair city one-hots were dropped in Part 7.
Recall	<code>TP / (TP + FN)</code> — of the items that <i>are</i> positive, how many were found.
Regularisation	Penalising weight size to reduce variance. Rejected here after measuring that it cost six times the error.
Residual	$y - \hat{y}$. Studying residuals is how Part 5 recovered the generating formula.
Ridge regression	Linear regression with an L2 penalty: $w = (X^TX + \alpha I)^{-1}X^Ty$.
RSS	Residual sum of squares, $\sum (y - \hat{y})^2$. A sum, so comparable only within one split.
Stacking	Using base models' outputs as a meta-model's inputs. This project's chained pipeline, built with out-of-fold predictions.
Stratified split	A split preserving each class's proportion in both halves. Used for all classification splits here.
Target / label	The column being predicted. Here <code>total_trip_cost</code> .
Train/serve mismatch	Training a model on inputs of a different quality from what it receives in production. Measured explicitly here — and it cost essentially nothing, because the sub-model errors are zero-mean.
Underfitting	Model too simple, or features too weak, to capture the relationship; poor on both splits. All five classifiers on the original dataset's traffic target.

Variance (of a model)	Sensitivity to which particular training rows were seen. High-variance models score well on train, poorly on test.
Winding factor	This project's term for road distance ÷ straight-line distance. Observed mean 1.2520, range 1.0437 to 3.6015 across 840 real routes — the variation a single constant cannot express.

How to reproduce every number in this report

16.1 Just run it

```
cd backend
pip install -r requirements.txt
uvicorn app:app                # -> http://127.0.0.1:8000    the API

cd ../frontend
npm ci
npm run dev                    # -> http://127.0.0.1:5173    the UI
```

Two processes locally, because there are two in production. Vite proxies `/api` to port 8000, so no backend URL is configured for local work and development exercises the same relative paths the deployed build does.

No training step is needed to serve a prediction: both model bundles and the two data files the API reads at import are committed beside it in `backend/`.

16.2 Rebuild everything from scratch, in order

Run from `ml/`. Note where the outputs land: a script that produces something the server loads writes it *into* `backend/`, and the distance table is written into `frontend/`. There is no copy step to forget, because there is no copy.

```
python scripts/build_cities.py          # 3,739 cities, real lat/lon -> backend/data/
python scripts/build_fuel_prices.py      # per-state fuel prices -> backend/data/
python scripts/build_wide_dataset.py     # 25,000 trips over real geography
python scripts/fetch_real_distances.py   # 840 REAL road distances from OSRM (~6 min)
python scripts/build_distance_matrix.py  # -> frontend/public/city_distances.json

python scripts/train_pipeline.py         # -> backend/models/pipeline.joblib (the sub-models)
python scripts/train_models.py          # -> backend/models/model.joblib (band + curve)

python scripts/evaluate_models.py        # -> ml/data/task5_evaluation.json (~5 min)
python scripts/build_report_pdf.py       # -> ml/Project_Report.pdf (this document)

cd ../backend && pytest -q               # 70 tests
```

Where the order is load-bearing, and where it is not. The data builders run first because everything downstream reads their output. `train_pipeline.py` and `train_models.py` are independent of each other — both fit from the CSV, neither loads the other's bundle — but `evaluate_models.py` reads `pipeline_report.json`, so it must follow the first of them, and `build_report_pdf.py` reads all three JSON reports, so it must be last. It re-runs its own two generated chapters on every build, so this document cannot quote a number older than the run that produced it.

16.3 Which file produces which number

Number in this report	Produced by
Parts 2–4 — the dataset and its statistics	<code>ml/road_trip_data.csv</code> , 1,140 rows. Every figure quoted is a pandas aggregate over that one file
Part 5 formula recovery	The recovered coefficients are the constants block at the head of <code>scripts/build_wide_dataset.py</code> — the formula is not merely reported, it is what regenerates the data
Part 6 dataset statistics	<code>scripts/build_wide_dataset.py</code>
Parts 7–9 — regression, interactions, ridge	<code>ml/data/regression_report.json</code> via <code>scripts/train_models.py</code>
Part 10 sub-model tables	<code>ml/data/pipeline_report.json</code> via <code>scripts/train_pipeline.py</code>
Parts 11–12 — classification and Task 5	<code>ml/data/task5_evaluation.json</code> via <code>scripts/evaluate_models.py</code>
The 840 real distances	<code>ml/data/real_distances.csv</code> via <code>scripts/fetch_real_distances.py</code>
Parts 10 and 12 of this PDF	<code>scripts/build_report_week10.py</code> and <code>scripts/build_report_tables.py</code> — generated from the JSON at build time, never typed

16.4 Reproducibility guarantees

- **Every random operation is seeded.** `random_state=42` for all splits and models; a fixed seed for route sampling and for the bootstrap.
- **Verified in practice.** After re-running the whole pipeline to fix the model file sizes, every metric returned *bit-identical* — R^2 0.981574, MAE ₹291.69, distance MAE 30.67 km.
- **One scoring module.** Every metric in this report comes from `scripts/evaluation.py` , and the two generated chapters read the JSON it writes, so this document and the API cannot quote different numbers for the same model.
- **One feature contract.** `backend/roadtrip_features.py` names each model's columns and their order once. The training scripts import that file out of `backend/` rather than keeping a copy, so a feature added for training cannot go missing at serve time — which is the failure the arrangement exists to prevent.
- **70 tests** pin the behaviour, including the error budget and the honest reporting.

Closing note

If only one thing from this report is remembered, make it this: the project reports **R^2 0.9816 and MAE ₹292** as its accuracy, not the R^2 0.9998 that appears in Parts 8 and 9. The larger number is real but was measured with the model holding the two terms that carry its own uncertainty. Knowing the difference — and being able to explain *why* the difference exists — is what the ten weeks were actually for.